

铁路图像质量感知深度识别软件源代码

软件名称：铁路图像质量感知深度识别软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. scripts/continue_overnight.sh
2. scripts/eval_faster_rcnn.py
3. scripts/eval_map.py
4. scripts/eval_ultralytics.py
5. scripts/eval.py
6. scripts/generate_sample_config.py
7. scripts/prepare_dataset.py
8. scripts/run_download_prepare.sh
9. scripts/run_formal_matrix.sh
10. scripts/run_ruod_degradation_eval.sh
11. scripts/train_faster_rcnn.py
12. scripts/train_ultralytics.py
13. scripts/train.py
14. src/__init__.py
15. src/datasets/__init__.py
16. src/datasets/underwater_detection.py
17. src/engine/__init__.py
18. src/engine/detection_ops.py
19. src/engine/trainer.py
20. src/external/__init__.py
21. src/external/faster_rcnn_runner.py
22. src/external/ultralytics_runner.py
23. src/losses/__init__.py
24. src/losses/detection_loss.py
25. src/models/__init__.py
26. src/models/modules/__init__.py
27. src/models/modules/cross_residual.py
28. src/models/modules/quality_aware_fusion.py
29. src/models/qdcr_net.py
30. src/utils/__init__.py
31. src/utils/config.py
32. src/utils/dingtalk_util.py
33. src/utils/experiment_notify.py
34. src/utils/logger.py
35. src/utils/tb_logger.py
36. src/utils/tracker.py

scripts/continue_overnight.sh

```
#!/usr/bin/env bash
# 使用 UTF-8 编写注释。
# 该脚本用于夜间续跑样例实验，并在 RUOD 压缩包准备好后自动生成配置。
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

LOG_DIR="$ROOT/outputs/automation"
mkdir -p "$LOG_DIR"

GPU_UTIL_THRESHOLD="${GPU_UTIL_THRESHOLD:-20}"
GPU_MEM_USED_THRESHOLD="${GPU_MEM_USED_THRESHOLD:-2048}"
GPU_POLL_SECONDS="${GPU_POLL_SECONDS:-300}"
RUOD_POLL_SECONDS="${RUOD_POLL_SECONDS:-300}"

timestamp() {
    date '+%F %T'
}

log() {
    # 所有信息统一追加到标准输出，并由 tee 写入对应日志。
    printf '[%s] %s\n' "$(timestamp)" "$*"
}

run_logged() {
    # 包装任意命令，自动记录任务名和输出日志。
    local name="$1"
    shift
    log "running: $name"
    "$@" 2>&1 | tee "$LOG_DIR/${name}.log"
}

wait_for_gpu() {
    # 同时检查 GPU 利用率与显存占用，避免和其他任务抢卡。
    if ! command -v nvidia-smi >/dev/null 2>&1; then
        log "nvidia-smi not found, skip GPU wait"
        return 0
    fi

    while true; do
        local query
        query="$(nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv,noheader,nounits | head -n1)"
        local util
        local mem
        util="$(printf '%s' "$query" | cut -d',' -f1 | tr -d ' ')"
        mem="$(printf '%s' "$query" | cut -d',' -f2 | tr -d ' ')"

        if [[ -n "$util" && -n "$mem" && "$util" -le "$GPU_UTIL_THRESHOLD" && "$mem" -le "$GPU_MEM_USED_THRESHOLD" ]];
        then
            log "gpu is available: util=${util}% mem=${mem}MiB"
            return 0
        fi

        log "gpu busy: util=${util:-unknown}% mem=${mem:-unknown}MiB, sleep ${GPU_POLL_SECONDS}s"
        sleep "$GPU_POLL_SECONDS"
    done
}

wait_for_valid_zip() {
    # 持续轮询 RUOD 压缩包，直到文件存在且 unzip 校验通过。
    local zip_path="$1"
    while true; do
        if [[ -s "$zip_path" ]] && unzip -tqq "$zip_path" >/dev/null 2>&1; then
            log "zip ready: $zip_path"
            return 0
        fi
    done
}
```

```

    log "zip not ready: $zip_path, sleep ${RUOD_POLL_SECONDS}s"
    sleep "$RUOD_POLL_SECONDS"
done
}

prepare_ruod() {
# RUOD 数据集到位后, 自动解压并生成 QDCR/Base 两套样例配置。
local zip_path="$ROOT/data/raw/RUOD.zip"
local extract_dir="$ROOT/data/datasets/downloads/RUOD"

wait_for_valid_zip "$zip_path"
mkdir -p "$extract_dir"

if [[ ! -d "$extract_dir/train" && ! -d "$extract_dir/valid" && ! -d "$extract_dir/val" ]]; then
    run_logged "ruod_unzip" unzip -o "$zip_path" -d "$extract_dir"
else
    log "skip unzip, dataset folders already exist in $extract_dir"
fi

run_logged \
    "ruod_qdcr_config" \
    conda run -n yolo python scripts/generate_sample_config.py \
    --dataset-root "$extract_dir" \
    --template configs/qdcr_net.yaml \
    --output configs/generated/qdcr_ruod_sample.yaml \
    --experiment-name qdcr_ruod_sample \
    --epochs 3 \
    --batch-size 4 \
    --max-batches 8 \
    --image-size 320 \
    --max-objects 8

run_logged \
    "ruod_base_config" \
    conda run -n yolo python scripts/generate_sample_config.py \
    --dataset-root "$extract_dir" \
    --template configs/base.yaml \
    --output configs/generated/base_ruod_sample.yaml \
    --experiment-name baseline_ruod_sample \
    --epochs 3 \
    --batch-size 4 \
    --max-batches 8 \
    --image-size 320 \
    --max-objects 8
}

run_sample_suite() {
# 样例实验统一遵循 “训练 -> 评估” 顺序。
local config_path="$1"
local stem="$2"
wait_for_gpu
run_logged "${stem}_train" conda run -n yolo python scripts/train.py --config "$config_path"
wait_for_gpu
run_logged "${stem}_eval" conda run -n yolo python scripts/eval_map.py --config "$config_path"
}

main() {
# 先继续现有 Brackish 样例, 再在 RUOD 数据准备好后补跑 RUOD。
log "overnight continuation started"

run_sample_suite "configs/generated/qdcr_brackish_sample.yaml" "qdcr_brackish_sample"
run_sample_suite "configs/generated/base_brackish_sample.yaml" "base_brackish_sample"

prepare_ruod
run_sample_suite "configs/generated/qdcr_ruod_sample.yaml" "qdcr_ruod_sample"
run_sample_suite "configs/generated/base_ruod_sample.yaml" "base_ruod_sample"

log "overnight continuation completed"
}

```

```
main "$@"
```

scripts/eval_faster_rcnn.py

```
# -*- coding: utf-8 -*-
"""Faster R-CNN 评估入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import FasterRCNNRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate Faster R-CNN detector.")
    parser.add_argument("--config", type=Path, required=True)
    return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.eval_faster_rcnn")
    args = parse_args()
    config = load_config(args.config)
    runner = FasterRCNNRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.evaluate()
        end_time = datetime.now()
        print("Evaluation metrics:")
        for key, value in metrics.items():
            print(f" {key}: {value:.6f}")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="评估", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval_faster_rcnn] evaluation failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="评估", error=exc)
        raise

if __name__ == "__main__":
    main()
```

scripts/eval_map.py

```
# -*- coding: utf-8 -*-
"""评估并打印 mAP 指标的入口脚本。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.engine.trainer import Trainer
```

```

from src.utils import get_logger
from src.utils.config import load_config
from src.utils.experiment_notify import send_experiment_notification

def parse_args() -> argparse.Namespace:
    """解析评估参数。"""
    parser = argparse.ArgumentParser(description="Evaluate a detection checkpoint and report mAP metrics.")
    parser.add_argument(
        "--config",
        type=Path,
        default=ROOT / "configs" / "qdc_r_net.yaml",
        help="Path to the YAML config file.",
    )
    return parser.parse_args()

def main() -> None:
    """运行评估并将指标打印到标准输出。"""
    logger = get_logger("qdc_r_eval_map")
    args = parse_args()
    config = load_config(args.config)
    trainer = Trainer(config)
    start_time = datetime.now()
    try:
        metrics = trainer.evaluate()
        end_time = datetime.now()
        print("Evaluation metrics:")
        for key, value in metrics.items():
            print(f" {key}: {value:.6f}")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=True,
            stage="评估",
            metrics=metrics,
        )
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval_map] evaluation failed")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=False,
            stage="评估",
            error=exc,
        )
    raise

if __name__ == "__main__":
    main()

```

scripts/eval_ultralytics.py

```

# -*- coding: utf-8 -*-
"""Ultralytics 模型评估入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

```

```

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import UltralyticsRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate Ultralytics detectors.")
    parser.add_argument("--config", type=Path, required=True)
    return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.eval_ultralytics")
    args = parse_args()
    config = load_config(args.config)
    runner = UltralyticsRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.evaluate()
        end_time = datetime.now()
        print("Evaluation metrics:")
        for key, value in metrics.items():
            print(f"  {key}: {value:.6f}")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="评估", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval_ultralytics] evaluation failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="评估", error=exc)
        raise

if __name__ == "__main__":
    main()

```

scripts/eval.py

```

# -*- coding: utf-8 -*-
"""评估入口脚本。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.engine.trainer import Trainer
from src.utils import get_logger
from src.utils.config import load_config
from src.utils.experiment_notify import send_experiment_notification

def parse_args() -> argparse.Namespace:
    """解析评估命令行参数。"""
    parser = argparse.ArgumentParser(description="Evaluate a detection model.")
    parser.add_argument(
        "--config",
        type=Path,
        default=ROOT / "configs" / "qdcrc_net.yaml",
        help="Path to the YAML config file.",
    )

```

```

    return parser.parse_args()

def main() -> None:
    """加载配置并执行评估。"""
    logger = get_logger("qdc.eval")
    args = parse_args()
    config = load_config(args.config)
    trainer = Trainer(config)
    start_time = datetime.now()
    try:
        metrics = trainer.evaluate()
        end_time = datetime.now()
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=True,
            stage="评估",
            metrics=metrics,
        )
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval] evaluation failed")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=False,
            stage="评估",
            error=exc,
        )
    raise

if __name__ == "__main__":
    main()

```

scripts/generate_sample_config.py

```

# -*- coding: utf-8 -*-
"""根据数据集目录自动生成样例实验配置。"""

import argparse
from pathlib import Path
import sys

import yaml

ROOT = Path(__file__).resolve().parents[1]

def parse_args() -> argparse.Namespace:
    """解析配置生成脚本参数。"""
    parser = argparse.ArgumentParser(description="Generate a sample experiment config from a dataset root.")
    parser.add_argument("--dataset-root", type=Path, required=True)
    parser.add_argument("--template", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--experiment-name", type=str, required=True)
    parser.add_argument("--image-size", type=int, default=320)
    parser.add_argument("--epochs", type=int, default=3)
    parser.add_argument("--batch-size", type=int, default=4)
    parser.add_argument("--max-batches", type=int, default=8)
    parser.add_argument("--max-objects", type=int, default=8)

```

```

return parser.parse_args()

def find_dir(root: Path, split_names: List[str], leaf_names: set[str]) -> Path | None:
    """在数据集目录里搜索 train/val 对应的图片目录。"""
    for path in root.rglob("*"):
        if not path.is_dir():
            continue
        lower = path.as_posix().lower()
        if path.name.lower() not in leaf_names:
            continue
        if any(f"/{split}/" in lower or lower.endswith(f"/{split}") for split in split_names):
            return path
    return None

def main() -> None:
    """读取模板后按数据集结构填充实验配置。"""
    args = parse_args()
    dataset_root = args.dataset_root.resolve()
    template = yaml.safe_load(args.template.read_text(encoding="utf-8"))

    data_yaml = dataset_root / "data.yaml"
    names = template["dataset"].get("class_names", [])
    num_classes = template["dataset"].get("num_classes", len(names))
    if data_yaml.exists():
        data = yaml.safe_load(data_yaml.read_text(encoding="utf-8"))
        names = list(data.get("names", names))
        num_classes = int(data.get("nc", len(names)))

    train_images = find_dir(dataset_root, ["train"], {"images", "image"})
    val_images = find_dir(dataset_root, ["val", "valid", "validation"], {"images", "image"})
    if train_images is None:
        raise SystemExit(f"Could not find train/images under {dataset_root}")
    if val_images is None:
        # 没有单独验证集时退化为复用训练集，便于快速连通流程。
        val_images = train_images

    cfg = template
    cfg["experiment"]["name"] = args.experiment_name
    cfg["experiment"]["output_dir"] = f"outputs/checkpoints/{args.experiment_name}"
    cfg["experiment"]["runs_dir"] = f"runs/{args.experiment_name}"
    cfg["dataset"]["train_root"] = str(train_images.relative_to(ROOT))
    cfg["dataset"]["val_root"] = str(val_images.relative_to(ROOT))
    cfg["dataset"]["enhanced_root"] = str(train_images.relative_to(ROOT))
    cfg["dataset"]["num_classes"] = num_classes
    cfg["dataset"]["class_names"] = names
    cfg["dataset"]["synthetic_size"] = 0
    cfg["dataset"]["max_objects"] = args.max_objects
    cfg["dataset"]["image_size"] = args.image_size
    cfg["train"]["epochs"] = args.epochs
    cfg["train"]["batch_size"] = args.batch_size
    cfg["train"]["max_batches_per_epoch"] = args.max_batches
    cfg["eval"]["max_batches"] = args.max_batches

    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")
    print(args.output)

if __name__ == "__main__":
    main()

```

scripts/prepare_dataset.py

```

# -*- coding: utf-8 -*-
"""数据集连通性检查脚本。"""

from pathlib import Path

```

```

import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.datasets import UnderwaterDetectionDataset
from src.utils.config import load_config

def main() -> None:
    """读取配置并打印当前数据集索引状态。"""
    config = load_config(ROOT / "configs" / "qdc_r_net.yaml")
    dataset_config = config["dataset"]
    dataset = UnderwaterDetectionDataset(
        image_root=ROOT / dataset_config["train_root"],
        annotation_root=(ROOT / dataset_config["train_root"]).parent / "labels",
        enhanced_root=ROOT / dataset_config["enhanced_root"],
        num_classes=dataset_config["num_classes"],
        image_size=dataset_config.get("image_size", 128),
        synthetic_size=dataset_config.get("synthetic_size", 16),
        max_objects=dataset_config.get("max_objects", 8),
    )
    source = dataset[0]["source"] if len(dataset) else "empty"
    # 这里主要用于快速确认真实数据是否被发现，否则会退合成样本。
    print(f"Dataset root: {(ROOT / dataset_config['train_root']).resolve()}")
    print(f"Samples indexed: {len(dataset)}")
    print(f"Source mode: {source}")
    if len(dataset):
        sample = dataset[0]
        print(f"Objects in first sample: {int(sample['gt_classes'].numel())}")
    print("Supported real-data format: image files with YOLO labels stored as matching .txt files.")

if __name__ == "__main__":
    main()

```

scripts/run_download_prepare.sh

```

#!/usr/bin/env bash
# 使用 UTF-8 编写注释。
# 该脚本负责下载公开数据集、解压，并基于模板生成样例配置。
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

# 统一准备原始下载目录、解压目录、日志目录和生成配置目录。
mkdir -p data/raw data/datasets/downloads data/logs configs/generated
LOG="data/logs/download_prepare.log"

log() {
    # 所有步骤都写入同一个日志文件，便于断点排查。
    printf "[%s] %s\n" "$(date '+%F %T')" "$*" | tee -a "$LOG"
}

download_extract_generate() {
    # 针对单个数据集执行“下载 -> 解压 -> 生成 QDCR/Base 配置”的完整流水线。
    local name="$1"
    local url="$2"
    local qdc_r_cfg="$3"
    local base_cfg="$4"
    log "Downloading ${name}"
    curl -L -C - -o "data/raw/${name}.zip" "$url" >>"$LOG" 2>&1
    log "Extracting ${name}"
    unzip -o "data/raw/${name}.zip" -d "data/datasets/downloads/${name}" >>"$LOG" 2>&1
    log "Generating configs for ${name}"
    conda run -n yolo python scripts/generate_sample_config.py \

```

```

--dataset-root "data/datasets/downloads/${name}" \
--template "configs/qdcr_net.yaml" \
--output "${qdcr_cfg}" \
--experiment-name "$(basename "${qdcr_cfg}" .yaml)" >>"$LOG" 2>&1
conda run -n yolo python scripts/generate_sample_config.py \
--dataset-root "data/datasets/downloads/${name}" \
--template "configs/base.yaml" \
--output "${base_cfg}" \
--experiment-name "$(basename "${base_cfg}" .yaml)" >>"$LOG" 2>&1
log "Finished ${name}"
}

# 当前默认下载 Brackish 与 RUOD 两个公开数据集。
download_extract_generate "Brackish" "***" "configs/generated/qdcr_brackish_sample.yaml"
"configs/generated/base_brackish_sample.yaml"
download_extract_generate "RUOD" "***" "configs/generated/qdcr_ruod_sample.yaml"
"configs/generated/base_ruod_sample.yaml"

```

scripts/run_formal_matrix.sh

```

#!/usr/bin/env bash
# 使用 UTF-8 编写注释。
# 该脚本顺序执行正式实验矩阵中的训练与评估任务。
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

LOG_DIR="$ROOT/outputs/automation"
mkdir -p "$LOG_DIR"

GPU_MEM_USED_THRESHOLD="${GPU_MEM_USED_THRESHOLD:-4500}"
GPU_POLL_SECONDS="${GPU_POLL_SECONDS:-180}"

DATASET_FILTER="${DATASET_FILTER:-}"
MODEL_FILTER="${MODEL_FILTER:-}"
CONFIG_GLOB="${CONFIG_GLOB:-configs/formal/*.yaml}"

timestamp() {
    date '+%F %T'
}

log() {
    printf "[%s] %s\n" "$(timestamp)" "$*"
}

wait_for_gpu() {
    # 如果机器支持 nvidia-smi, 则等待显存占用下降到阈值以下后再启动任务。
    if ! command -v nvidia-smi >/dev/null 2>&1; then
        return 0
    fi

    while true; do
        local used
        used="$(nvidia-smi --query-gpu=memory.used --format=csv,noheader,nounits | head -n1 | tr -d ' ')"
        if [[ -n "$used" && "$used" -le "$GPU_MEM_USED_THRESHOLD" ]]; then
            log "gpu ready: memory.used=${used}MiB"
            return 0
        fi
        log "gpu busy: memory.used=${used:-unknown}MiB, sleep ${GPU_POLL_SECONDS}s"
        sleep "$GPU_POLL_SECONDS"
    done
}

run_one() {
    # 单个配置先训练再评估, 并把日志分别落到自动化输出目录。
    local config_path="$1"
    local stem
    local model_name
}

```

```

local train_cmd
local eval_cmd
stem="$(basename "$config_path" .yaml)"
model_name="$(python - <<PY
import yaml
from pathlib import Path
cfg = yaml.safe_load(Path("$config_path").read_text())
print(str(cfg.get("model", {}).get("name", "qdcn_net"))))
PY
)"

case "$model_name" in
    yolov8|rtddetr|underwater_enhance_yolov8)
        train_cmd=(python scripts/train_ultralytics.py --config "$config_path")
        eval_cmd=(python scripts/eval_ultralytics.py --config "$config_path")
        ;;
    faster_rcnn)
        train_cmd=(python scripts/train_faster_rcnn.py --config "$config_path")
        eval_cmd=(python scripts/eval_faster_rcnn.py --config "$config_path")
        ;;
    *)
        train_cmd=(python scripts/train.py --config "$config_path")
        eval_cmd=(python scripts/eval_map.py --config "$config_path")
        ;;
esac

wait_for_gpu
log "train start: $config_path model=${model_name}"
conda run -n yolo "${train_cmd[@]}" 2>&1 | tee "$LOG_DIR/${stem}_train.log"

wait_for_gpu
log "eval start: $config_path model=${model_name}"
conda run -n yolo "${eval_cmd[@]}" 2>&1 | tee "$LOG_DIR/${stem}_eval.log"
}

should_run() {
    local config_path="$1"
    local stem
    stem="$(basename "$config_path" .yaml)"
    if [[ -n "$DATASET_FILTER" && ! "$stem" =~ $DATASET_FILTER ]]; then
        return 1
    fi
    if [[ -n "$MODEL_FILTER" && ! "$stem" =~ $MODEL_FILTER ]]; then
        return 1
    fi
    return 0
}

collect_configs() {
    local config_path
    for config_path in $CONFIG_GLOB; do
        [[ -f "$config_path" ]] || continue
        should_run "$config_path" || continue
        printf '%s\n' "$config_path"
    done | sort
}

main() {
    # 默认自动发现 configs/formal/*.yaml, 后续新增模型配置后脚本无需再次硬编码。
    # 示例:
    # MODEL_FILTER='^(base|qdcn)_' bash scripts/run_formal_matrix.sh
    # DATASET_FILTER='ruod' bash scripts/run_formal_matrix.sh
    # MODEL_FILTER='^(yolov8|rtddetr)_' DATASET_FILTER='brackish' bash scripts/run_formal_matrix.sh
    mapfile -t configs < <(collect_configs)
    if [[ "${#configs[@]}" -eq 0 ]]; then
        log "no formal configs matched; CONFIG_GLOB=${CONFIG_GLOB} DATASET_FILTER=${DATASET_FILTER:-<empty>}"
        MODEL_FILTER=${MODEL_FILTER:-<empty>}"
        exit 1
    fi

    log "formal matrix runner started"
}

```

```

log "selected configs (${#configs[@]}): ${configs[*]}"
for config_path in "${configs[@]"; do
    run_one "$config_path"
done
log "formal matrix runner completed"
}

main "$@"

```

scripts/run_ruod_degradation_eval.sh

```

#!/usr/bin/env bash
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

LOG_DIR="$ROOT/outputs/automation"
mkdir -p "$LOG_DIR"

CONFIGS=(
    "configs/formal/base_ruod_testb_eval.yaml"
    "configs/formal/base_ruod_testc_eval.yaml"
    "configs/formal/base_ruod_testl_eval.yaml"
    "configs/formal/qdcr_ruod_testb_eval.yaml"
    "configs/formal/qdcr_ruod_testc_eval.yaml"
    "configs/formal/qdcr_ruod_testl_eval.yaml"
)

timestamp() {
    date '+%F %T'
}

log() {
    printf "[%s] %s\n" "$(timestamp)" "$*"
}

main() {
    log "ruod degradation evaluation started"
    for config_path in "${CONFIGS[@]"; do
        stem="$(basename "$config_path" .yaml)"
        log "eval start: $config_path"
        conda run -n yolo python scripts/eval_map.py --config "$config_path" 2>&1 | tee "$LOG_DIR/${stem}.log"
    done
    log "ruod degradation evaluation completed"
}

main "$@"

```

scripts/train_faster_rcnn.py

```

# -*- coding: utf-8 -*-
"""Faster R-CNN 训练入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import FasterRCNNRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:

```

```

parser = argparse.ArgumentParser(description="Train Faster R-CNN detector.")
parser.add_argument("--config", type=Path, required=True)
return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.train_faster_rcnn")
    args = parse_args()
    config = load_config(args.config)
    runner = FasterRCNNRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.train()
        end_time = datetime.now()
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="训练", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[train_faster_rcnn] training failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="训练", error=exc)
        raise

if __name__ == "__main__":
    main()

```

scripts/train_ultralytics.py

```

# -*- coding: utf-8 -*-
"""Ultralytics 模型训练入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import UltralyticsRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train Ultralytics detectors.")
    parser.add_argument("--config", type=Path, required=True)
    return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.train_ultralytics")
    args = parse_args()
    config = load_config(args.config)
    runner = UltralyticsRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.train()
        end_time = datetime.now()
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="训练", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[train_ultralytics] training failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="训练", error=exc)
        raise

```

```
if __name__ == "__main__":
    main()
```

scripts/train.py

```
# -*- coding: utf-8 -*-
"""训练入口脚本。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.engine.trainer import Trainer
from src.utils import get_logger
from src.utils.config import load_config
from src.utils.experiment_notify import send_experiment_notification

def parse_args() -> argparse.Namespace:
    """解析训练命令行参数。"""
    parser = argparse.ArgumentParser(description="Train a detection model.")
    parser.add_argument(
        "--config",
        type=Path,
        default=ROOT / "configs" / "qdcrcr_net.yaml",
        help="Path to the YAML config file.",
    )
    return parser.parse_args()

def main() -> None:
    """加载配置并启动训练流程。"""
    logger = get_logger("qdcrcr.train")
    args = parse_args()
    config = load_config(args.config)
    trainer = Trainer(config)
    start_time = datetime.now()
    try:
        summary = trainer.fit()
        end_time = datetime.now()
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=True,
            stage="训练",
            summary=summary,
        )
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[train] training failed")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=False,
            stage="训练",
            error=exc,
```

```
)
raise

if __name__ == "__main__":
    main()
```

src/__init__.py

```
# -*- coding: utf-8 -*-
"""QDCR-Net 源码主包。"""
```

src/datasets/__init__.py

```
# -*- coding: utf-8 -*-
"""数据集模块导入口。"""

from .underwater_detection import UnderwaterDetectionDataset

__all__ = ["UnderwaterDetectionDataset"]
```

src/datasets/underwater_detection.py

```
# -*- coding: utf-8 -*-
"""水下目标检测数据集定义。"""

from dataclasses import dataclass, field
import hashlib
from pathlib import Path
import tempfile
from typing import Any

from PIL import Image
import torch
from torch.utils.data import Dataset
from torchvision import transforms

@dataclass
class UnderwaterDetectionDataset(Dataset):
    """支持真实 YOLO 标注与合成回退样本的数据集。"""

    image_root: Path
    annotation_root: Path | None = None
    enhanced_root: Path | None = None
    num_classes: int = 4
    image_size: int = 128
    synthetic_size: int = 32
    max_objects: int = 8
    cache_dir: Path | None = None
    _samples: list[dict[str, Any]] = field(init=False, repr=False, default_factory=list)

    def __post_init__(self) -> None:
        """初始化路径、图像变换，并优先索引真实数据。"""
        self.image_root = Path(self.image_root)
        self.annotation_root = Path(self.annotation_root) if self.annotation_root else None
        self.enhanced_root = Path(self.enhanced_root) if self.enhanced_root else None
        self.cache_dir = Path(self.cache_dir) if self.cache_dir else None
        self.transform = transforms.Compose(
            [
                transforms.Resize((self.image_size, self.image_size)),
                transforms.ToTensor(),
            ]
        )

        image_files = self._discover_images(self.image_root)
```

```

    if image_files:
        for image_path in image_files:
            sample = self._build_real_sample(image_path)
            if sample is not None:
                self._samples.append(sample)

    if self._samples:
        return

    # 当本地没有可用真实数据时, 自动构造合成样本以保证训练链路可跑通。
    self._samples = [self._build_synthetic_sample(index) for index in range(self.synthetic_size)]

def __len__(self) -> int:
    return len(self._samples)

def __getitem__(self, index: int) -> dict[str, Any]:
    """按索引返回一个样本, 统一输出原图、增强图和标注。"""
    sample = self._samples[index]
    cached = self._load_cached_pair(sample)
    if cached is not None:
        raw_image, enhanced_image = cached
    elif sample["source"] == "real":
        raw_image, enhanced_image = self._build_real_pair(sample)
    else:
        # 合成样本直接动态生成一对原图/增强图, 避免额外落盘。
        raw_image, enhanced_image = self._build_synthetic_pair(sample["sample_id"], sample["annotations"])

    gt_boxes = torch.tensor(
        [annotation["bbox"] for annotation in sample["annotations"]],
        dtype=torch.float32,
    )
    gt_classes = torch.tensor(
        [annotation["class_id"] for annotation in sample["annotations"]],
        dtype=torch.long,
    )
    return {
        "sample_id": sample["sample_id"],
        "raw_image": raw_image,
        "enhanced_image": enhanced_image,
        "gt_boxes": gt_boxes,
        "gt_classes": gt_classes,
        "source": sample["source"],
    }

def prepare_cache(self, cache_dir: Path | None = None) -> None:
    """在训练前把图像变换结果预写到临时目录, 减少运行时 CPU 开销。"""
    target_dir = Path(cache_dir) if cache_dir is not None else self.cache_dir
    if target_dir is None:
        target_dir = Path(tempfile.gettempdir()) / "qdcrr_preprocessed" / self._cache_namespace()
    target_dir.mkdir(parents=True, exist_ok=True)
    self.cache_dir = target_dir

    for sample in self._samples:
        cache_path = self._cache_path(sample)
        if cache_path is None or cache_path.exists():
            continue
        if sample["source"] == "real":
            raw_image, enhanced_image = self._build_real_pair(sample)
        else:
            raw_image, enhanced_image = self._build_synthetic_pair(sample["sample_id"], sample["annotations"])
        torch.save(
            {
                "raw_image": raw_image.contiguous(),
                "enhanced_image": enhanced_image.contiguous(),
            },
            cache_path,
        )

def _discover_images(self, root: Path) -> List[Path]:
    """递归发现数据目录里的常见图像文件。"""
    if not root.exists():

```

```

        return []

    image_files: list[Path] = []
    for pattern in ("*.jpg", "*.jpeg", "*.png", "*.bmp"):
        image_files.extend(sorted(root.rglob(pattern)))
    return image_files

def _build_real_sample(self, image_path: Path) -> dict[str, Any] | None:
    """根据图像路径组装真实样本，如果无标注则跳过。"""
    annotations = self._load_labels_for_image(image_path)
    if not annotations:
        return None

    # 优先保留面积更大的目标，避免超出 max_objects 时随机截断。
    annotations.sort(key=lambda item: item["bbox"][2] * item["bbox"][3], reverse=True)
    return {
        "sample_id": image_path.relative_to(self.image_root).as_posix(),
        "annotations": annotations[: self.max_objects],
        "image_path": image_path,
        "enhanced_path": self._resolve_enhanced_path(image_path),
        "source": "real",
    }

def _build_synthetic_sample(self, index: int) -> dict[str, Any]:
    """构造一个可重复生成的合成样本描述。"""
    object_count = 1 + (index % min(3, self.max_objects))
    annotations = []
    for offset in range(object_count):
        label = (index + offset) % self.num_classes
        annotations.append(
            {
                "class_id": label,
                "bbox": self._synthetic_bbox(label, offset),
            }
        )
    return {
        "sample_id": f"synthetic-{index}",
        "annotations": annotations,
        "source": "synthetic",
    }

def _load_labels_for_image(self, image_path: Path) -> list[dict[str, Any]]:
    """读取 YOLO 标签文件，并过滤非法类别或越界框。"""
    label_path = self._resolve_label_path(image_path)
    if label_path is None or not label_path.exists():
        return []

    labels: list[dict[str, Any]] = []
    for raw_line in label_path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line:
            continue

        parts = line.split()
        if len(parts) < 5:
            continue

        try:
            class_id = int(float(parts[0]))
            bbox = [float(value) for value in parts[1:5]]
        except ValueError:
            continue

        if 0 <= class_id < self.num_classes and all(0.0 <= value <= 1.0 for value in bbox):
            labels.append({"class_id": class_id, "bbox": bbox})

    return labels

def _resolve_label_path(self, image_path: Path) -> Path | None:
    """尝试从多种常见目录结构推断标签路径。"""
    candidates: list[Path] = []

```

```

if self.annotation_root is not None:
    try:
        candidates.append(
            self.annotation_root / image_path.relative_to(self.image_root).with_suffix(".txt")
        )
    except ValueError:
        candidates.append(self.annotation_root / f"{image_path.stem}.txt")

candidates.append(image_path.with_suffix(".txt"))

parts = list(image_path.parts)
for marker in ("images", "image", "imgs", "img"):
    if marker in parts:
        marker_index = parts.index(marker)
        # 常见的 YOLO 数据集会把 images 目录平移为 labels 目录。
        remapped = Path(*parts[:marker_index], "labels", *parts[marker_index + 1 :])
        candidates.append(remapped.with_suffix(".txt"))

for candidate in candidates:
    if candidate.exists():
        return candidate
return candidates[0] if candidates else None

def _resolve_enhanced_path(self, image_path: Path) -> Path | None:
    """根据增强图根目录推断对应增强图位置。"""
    if self.enhanced_root is None or not self.enhanced_root.exists():
        return None

    candidates: list[Path] = []
    try:
        candidates.append(self.enhanced_root / image_path.relative_to(self.image_root))
    except ValueError:
        candidates.append(self.enhanced_root / image_path.name)

    for candidate in candidates:
        if candidate.exists():
            return candidate
    return None

def _load_image_tensor(self, path: Path) -> torch.Tensor:
    """读取单张图像并转换为模型输入张量。"""
    image = Image.open(path).convert("RGB")
    return self.transform(image)

def _build_real_pair(self, sample: dict[str, Any]) -> tuple[torch.Tensor, torch.Tensor]:
    raw_image = self._load_image_tensor(sample["image_path"])
    enhanced_image = (
        self._load_image_tensor(sample["enhanced_path"])
        if sample["enhanced_path"] is not None
        else self._auto_enhance(raw_image)
    )
    return raw_image, enhanced_image

def _load_cached_pair(self, sample: dict[str, Any]) -> tuple[torch.Tensor, torch.Tensor] | None:
    cache_path = self._cache_path(sample)
    if cache_path is None or not cache_path.exists():
        return None
    payload = torch.load(cache_path, map_location="cpu")
    return payload["raw_image"], payload["enhanced_image"]

def _cache_path(self, sample: dict[str, Any]) -> Path | None:
    if self.cache_dir is None:
        return None
    cache_key = hashlib.sha256(sample["sample_id"].encode("utf-8")).hexdigest()
    return self.cache_dir / f"{cache_key}.pt"

def _cache_namespace(self) -> str:
    signature = "|".join(
        [
            str(self.image_root),
            str(self.enhanced_root),

```

```

        str(self.image_size),
        str(self.max_objects),
        str(len(self._samples)),
    ]
)
return hashlib.sha256(signature.encode("utf-8")).hexdigest()[:16]

def _auto_enhance(self, image_tensor: torch.Tensor) -> torch.Tensor:
    """当缺少增强图时，使用简单亮度/对比度提升做兜底。"""
    return torch.clamp(image_tensor * 1.1 + 0.03, 0.0, 1.0)

def _build_synthetic_pair(
    self,
    sample_id: str,
    annotations: list[dict[str, Any]],
) -> tuple[torch.Tensor, torch.Tensor]:
    """根据样本 ID 稳定生成一对合成图像。"""
    seed = int(hashlib.sha256(sample_id.encode("utf-8")).hexdigest()[:8], 16)
    generator = torch.Generator().manual_seed(seed)
    raw = torch.rand((3, self.image_size, self.image_size), generator=generator) * 0.12

    for annotation in annotations:
        x1, y1, x2, y2 = self._xywh_to_xyxy(annotation["bbox"])
        left = max(int(x1 * self.image_size), 0)
        top = max(int(y1 * self.image_size), 0)
        right = min(int(x2 * self.image_size), self.image_size)
        bottom = min(int(y2 * self.image_size), self.image_size)
        channel = int(annotation["class_id"]) % 3
        # 不同类别映射到不同通道，便于让模型看到稳定的类别差异。
        raw[channel, top:bottom, left:right] += 0.75

    raw = torch.clamp(raw, 0.0, 1.0)
    enhanced = torch.clamp(raw * 1.15 + 0.05, 0.0, 1.0)
    return raw, enhanced

def _synthetic_bbox(self, label: int, slot: int) -> list[float]:
    """生成一个位于合理范围内的归一化边界框。"""
    width = max(0.14, 0.26 - 0.03 * slot)
    height = max(0.12, 0.22 - 0.02 * slot)
    center_x = min(0.18 + 0.18 * label + 0.08 * slot, 0.82)
    center_y = min(0.20 + 0.12 * label + 0.10 * slot, 0.82)
    return [center_x, center_y, width, height]

def _xywh_to_xyxy(self, bbox: list[float]) -> tuple[float, float, float, float]:
    """把中心点格式边界框转换为左上角/右下角格式。"""
    center_x, center_y, width, height = bbox
    return (
        center_x - width / 2.0,
        center_y - height / 2.0,
        center_x + width / 2.0,
        center_y + height / 2.0,
    )

```

src/engine/__init__.py

```

# -*- coding: utf-8 -*-
"""训练与检测算子模块导出入口。"""

from .trainer import Trainer
from .detection_ops import compute_detection_metrics, decode_predictions, greedy_match

__all__ = ["Trainer", "compute_detection_metrics", "decode_predictions", "greedy_match"]

```

src/engine/detection_ops.py

```

# -*- coding: utf-8 -*-
"""检测任务中的匹配、解码和指标计算工具。"""

```

```

import torch
from torchvision.ops import nms

def pairwise_iou_xywh(boxes_a: torch.Tensor, boxes_b: torch.Tensor) -> torch.Tensor:
    """计算两组 xywh 归一化框之间的两两 IoU。"""
    if boxes_a.numel() == 0 or boxes_b.numel() == 0:
        return torch.zeros((boxes_a.size(0), boxes_b.size(0)), dtype=boxes_a.dtype, device=boxes_a.device)

    boxes_b = boxes_b.to(device=boxes_a.device, dtype=boxes_a.dtype)

    boxes_a_xyxy = xywh_to_xyxy(boxes_a)
    boxes_b_xyxy = xywh_to_xyxy(boxes_b)

    left_top = torch.maximum(boxes_a_xyxy[:, None, :2], boxes_b_xyxy[None, :, :2])
    right_bottom = torch.minimum(boxes_a_xyxy[:, None, 2:], boxes_b_xyxy[None, :, 2:])
    wh = (right_bottom - left_top).clamp(min=0.0)
    intersection = wh[..., 0] * wh[..., 1]

    area_a = (boxes_a_xyxy[:, 2] - boxes_a_xyxy[:, 0]).clamp(min=0.0) * (
        boxes_a_xyxy[:, 3] - boxes_a_xyxy[:, 1]
    ).clamp(min=0.0)
    area_b = (boxes_b_xyxy[:, 2] - boxes_b_xyxy[:, 0]).clamp(min=0.0) * (
        boxes_b_xyxy[:, 3] - boxes_b_xyxy[:, 1]
    ).clamp(min=0.0)
    union = area_a[:, None] + area_b[None, :] - intersection
    return intersection / union.clamp(min=1e-6)

def xywh_to_xyxy(boxes: torch.Tensor) -> torch.Tensor:
    """把中心点格式边界框转换为角点格式。"""
    center_x = boxes[..., 0]
    center_y = boxes[..., 1]
    width = boxes[..., 2]
    height = boxes[..., 3]
    return torch.stack(
        [
            center_x - width / 2.0,
            center_y - height / 2.0,
            center_x + width / 2.0,
            center_y + height / 2.0,
        ],
        dim=-1,
    )

def greedy_match(
    logits: torch.Tensor,
    pred_boxes: torch.Tensor,
    gt_classes: torch.Tensor,
    gt_boxes: torch.Tensor,
    background_class: int,
    cls_weight: float = 1.0,
    l1_weight: float = 3.0,
    iou_weight: float = 3.0,
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """基于分类、L1 与 IoU 混合代价做贪心匹配。"""
    num_queries = logits.size(0)
    matched_classes = torch.full(
        (num_queries,),
        background_class,
        dtype=torch.long,
        device=logits.device,
    )
    matched_boxes = torch.zeros((num_queries, 4), dtype=pred_boxes.dtype, device=pred_boxes.device)
    matched_mask = torch.zeros((num_queries,), dtype=torch.bool, device=logits.device)

    if gt_classes.numel() == 0:
        return matched_classes, matched_boxes, matched_mask

    probabilities = logits.softmax(dim=-1)

```

```

iou_matrix = pairwise_iou_xywh(pred_boxes, gt_boxes)
l1_matrix = torch.cdist(pred_boxes, gt_boxes, p=1)
cls_cost = -probabilities[:, gt_classes]
total_cost = cls_weight * cls_cost + l1_weight * l1_matrix + iou_weight * (1.0 - iou_matrix)

```

```

remaining_cost = total_cost.clone()
max_matches = min(num_queries, gt_classes.size(0))
inf = torch.tensor(torch.inf, device=remaining_cost.device, dtype=remaining_cost.dtype)
for _ in range(max_matches):
    flat_index = torch.argmax(remaining_cost).item()
    pred_index = flat_index // gt_classes.size(0)
    gt_index = flat_index % gt_classes.size(0)
    if not torch.isfinite(remaining_cost[pred_index, gt_index]):
        break
    matched_classes[pred_index] = gt_classes[gt_index]
    matched_boxes[pred_index] = gt_boxes[gt_index]
    matched_mask[pred_index] = True
    remaining_cost[pred_index, :] = inf
    remaining_cost[:, gt_index] = inf

return matched_classes, matched_boxes, matched_mask

```

```

def decode_predictions(
    logits: torch.Tensor,
    pred_boxes: torch.Tensor,
    background_class: int,
    conf_thresh: float,
    iou_thresh: float,
) -> list[dict[str, torch.Tensor]]:
    """把模型输出解码为按图片组织的检测结果，并做按类 NMS。"""
    batch_detections: list[dict[str, torch.Tensor]] = []
    probabilities = logits.softmax(dim=-1)
    scores, classes = probabilities[..., :-1].max(dim=-1)

    for batch_index in range(logits.size(0)):
        score = scores[batch_index]
        cls = classes[batch_index]
        box = pred_boxes[batch_index]
        keep = score >= conf_thresh
        score = score[keep]
        cls = cls[keep]
        box = box[keep]
        if score.numel() == 0:
            batch_detections.append(
                {
                    "scores": torch.zeros((0,), device=logits.device),
                    "classes": torch.zeros((0,), dtype=torch.long, device=logits.device),
                    "boxes": torch.zeros((0, 4), device=logits.device),
                }
            )
            continue

        kept_indices = []
        box_xyxy = xywh_to_xyxy(box)
        for class_id in cls.unique():
            # 按类别分别执行 NMS，避免不同类别之间相互抑制。
            class_mask = cls == class_id
            class_indices = torch.nonzero(class_mask, as_tuple=False).squeeze(1)
            selected = nms(box_xyxy[class_mask], score[class_mask], iou_thresh)
            kept_indices.append(class_indices[selected])
        final_indices = (
            torch.cat(kept_indices, dim=0)
            if kept_indices
            else torch.zeros((0,), dtype=torch.long, device=logits.device)
        )
        final_indices = final_indices[score[final_indices].argsort(descending=True)]
        batch_detections.append(
            {
                "scores": score[final_indices],
                "classes": cls[final_indices],

```

```

        "boxes": box[final_indices],
    }
)

return batch_detections

def compute_detection_metrics(
    predictions: list[dict[str, torch.Tensor]],
    ground_truths: list[dict[str, torch.Tensor]],
    num_classes: int,
) -> dict[str, float]:
    """计算 mAP、precision、recall 等聚合指标。"""
    ap50_per_class = []
    ap5095_per_class = []
    precisions = []
    recalls = []
    for class_id in range(num_classes):
        ap50 = _average_precision(predictions, ground_truths, class_id, 0.5)
        ap_thresholds = [
            _average_precision(predictions, ground_truths, class_id, threshold)
            for threshold in (0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95)
        ]
        precision, recall = _precision_recall(predictions, ground_truths, class_id, 0.5)
        ap50_per_class.append(ap50)
        ap5095_per_class.append(sum(ap_thresholds) / len(ap_thresholds))
        precisions.append(precision)
        recalls.append(recall)

    return {
        "map50": sum(ap50_per_class) / max(len(ap50_per_class), 1),
        "map50_95": sum(ap5095_per_class) / max(len(ap5095_per_class), 1),
        "precision": sum(precisions) / max(len(precisions), 1),
        "recall": sum(recalls) / max(len(recalls), 1),
    }

def _average_precision(
    predictions: list[dict[str, torch.Tensor]],
    ground_truths: list[dict[str, torch.Tensor]],
    class_id: int,
    iou_threshold: float,
) -> float:
    """针对单一类别和单一 IoU 阈值计算 AP。"""
    scored_predictions = []
    total_ground_truth = 0
    gt_lookup: dict[int, dict[str, torch.Tensor]] = {}

    for image_index, gt in enumerate(ground_truths):
        gt_mask = gt["classes"] == class_id
        gt_boxes = gt["boxes"][gt_mask]
        gt_lookup[image_index] = {
            "boxes": gt_boxes,
            "matched": torch.zeros((gt_boxes.size(0),), dtype=torch.bool),
        }
        total_ground_truth += int(gt_boxes.size(0))

    for image_index, prediction in enumerate(predictions):
        pred_mask = prediction["classes"] == class_id
        for score, box in zip(prediction["scores"][pred_mask], prediction["boxes"][pred_mask]):
            scored_predictions.append((float(score.item()), image_index, box))

    if total_ground_truth == 0:
        return 0.0

    scored_predictions.sort(key=lambda item: item[0], reverse=True)
    true_positive = []
    false_positive = []

    for _, image_index, pred_box in scored_predictions:
        gt_entry = gt_lookup[image_index]

```

```

gt_boxes = gt_entry["boxes"]
if gt_boxes.numel() == 0:
    true_positive.append(0.0)
    false_positive.append(1.0)
    continue

ious = pairwise_iou_xywh(pred_box.unsqueeze(0), gt_boxes).squeeze(0)
best_iou, best_index = torch.max(ious, dim=0)
if best_iou.item() >= iou_threshold and not gt_entry["matched"][best_index]:
    gt_entry["matched"][best_index] = True
    true_positive.append(1.0)
    false_positive.append(0.0)
else:
    true_positive.append(0.0)
    false_positive.append(1.0)

tp_cumsum = torch.tensor(true_positive).cumsum(dim=0)
fp_cumsum = torch.tensor(false_positive).cumsum(dim=0)
recalls = tp_cumsum / max(total_ground_truth, 1)
precisions = tp_cumsum / (tp_cumsum + fp_cumsum).clamp(min=1e-6)

recalls = torch.cat([torch.tensor([0.0]), recalls, torch.tensor([1.0])])
precisions = torch.cat([torch.tensor([1.0]), precisions, torch.tensor([0.0])])
# 从后向前做 precision envelope, 得到标准 PR 曲线面积。
for index in range(precisions.numel() - 2, -1, -1):
    precisions[index] = torch.maximum(precisions[index], precisions[index + 1])

recall_steps = recalls[1:] - recalls[:-1]
return float(torch.sum(recall_steps * precisions[1:]).item())

```

```

def _precision_recall(
    predictions: list[dict[str, torch.Tensor]],
    ground_truths: list[dict[str, torch.Tensor]],
    class_id: int,
    iou_threshold: float,
) -> tuple[float, float]:
    """在给定 IoU 阈值下计算单类别 precision 与 recall。"""
    scored_predictions = []
    total_ground_truth = 0
    gt_lookup: dict[int, dict[str, torch.Tensor]] = {}

    for image_index, gt in enumerate(ground_truths):
        gt_mask = gt["classes"] == class_id
        gt_boxes = gt["boxes"][gt_mask]
        gt_lookup[image_index] = {
            "boxes": gt_boxes,
            "matched": torch.zeros((gt_boxes.size(0),), dtype=torch.bool),
        }
        total_ground_truth += int(gt_boxes.size(0))

    for image_index, prediction in enumerate(predictions):
        pred_mask = prediction["classes"] == class_id
        for score, box in zip(prediction["scores"][pred_mask], prediction["boxes"][pred_mask]):
            scored_predictions.append((float(score.item()), image_index, box))

    scored_predictions.sort(key=lambda item: item[0], reverse=True)
    tp = 0
    fp = 0
    for _, image_index, pred_box in scored_predictions:
        gt_entry = gt_lookup[image_index]
        gt_boxes = gt_entry["boxes"]
        if gt_boxes.numel() == 0:
            fp += 1
            continue

        ious = pairwise_iou_xywh(pred_box.unsqueeze(0), gt_boxes).squeeze(0)
        best_iou, best_index = torch.max(ious, dim=0)
        if best_iou.item() >= iou_threshold and not gt_entry["matched"][best_index]:
            gt_entry["matched"][best_index] = True
            tp += 1

```

```

    else:
        fp += 1

precision = tp / max(tp + fp, 1)
recall = tp / max(total_ground_truth, 1)
return precision, recall

```

src/engine/trainer.py

```

# -*- coding: utf-8 -*-
"""训练器实现，负责训练、评估和实验产物写出。"""

from dataclasses import dataclass
import json
from pathlib import Path
import random
import tempfile
import time
from typing import Any

import torch
from torch import nn
from torch.utils.data import DataLoader
import yaml

from src.datasets import UnderwaterDetectionDataset
from src.engine.detection_ops import compute_detection_metrics, decode_predictions, greedy_match, pairwise_iou_xywh
from src.losses import DetectionLoss
from src.models import BaselineDetector, QDCRNet
from src.utils import ExperimentTracker, get_logger

@dataclass
class Trainer:
    """围绕配置驱动的最小训练/评估控制器。"""

    config: dict

    def __post_init__(self) -> None:
        """解析实验路径、运行设备并设置随机种子。"""
        self.logger = get_logger("qdcr.trainer")
        experiment_config = self.config.get("experiment", {})
        self.output_dir = Path(experiment_config.get("output_dir", "outputs/checkpoints/qdcr_net_smoke"))
        self.checkpoint_path = self.output_dir / "latest.pt"
        self.best_checkpoint_path = self.output_dir / "best.pt"
        self.prediction_path = self.output_dir / "predictions.json"
        self.metrics_path = self.output_dir / "metrics.json"
        self.config_snapshot_path = self.output_dir / "experiment_config.yaml"
        self.summary_path = self.output_dir / "experiment_summary.json"
        self.device = torch.device(
            experiment_config.get(
                "device",
                "cuda" if torch.cuda.is_available() else "cpu",
            )
        )
        self.run_dir: Path | None = None
        self.last_fit_summary: dict[str, Any] | None = None
        self._set_seed(int(experiment_config.get("seed", 42)))

    def fit(self) -> None:
        """执行完整训练流程，并在每轮结束后做验证与早停判断。"""
        train_dataset, val_dataset = self._build_datasets()
        model = self._build_model().to(self.device)
        loss_fn = DetectionLoss().to(self.device)
        train_config = self.config.get("train", {})
        eval_config = self.config.get("eval", {})
        early_stopping = train_config.get("early_stopping", {})
        optimizer = self._build_optimizer(model)
        tracker = ExperimentTracker(self.config)

```

```

self.run_dir = tracker.run_dir
use_amp = self.device.type == "cuda" and bool(train_config.get("amp", True))
scaler = torch.cuda.amp.GradScaler(enabled=use_amp)
num_workers = max(0, int(self.config.get("dataset", {}).get("workers", 0)))
pin_memory = self.device.type == "cuda"
persistent_workers = num_workers > 0
prefetch_factor = max(1, int(self.config.get("dataset", {}).get("prefetch_factor", 2)))
log_graph = bool(train_config.get("log_graph", False))
log_histograms = bool(train_config.get("log_histograms", False))
eval_interval = max(1, int(train_config.get("eval_interval", 5)))
batch_size = min(int(train_config.get("batch_size", 4)), len(train_dataset))
eval_batch_size = min(int(train_config.get("batch_size", 4)), len(val_dataset))
max_batches = int(train_config.get("max_batches_per_epoch", 2))
eval_max_batches = int(eval_config.get("max_batches", 2))
epochs = int(train_config.get("epochs", 1))
patience = int(early_stopping.get("patience", 5))
min_delta = float(early_stopping.get("min_delta", 1e-4))
monitor = str(early_stopping.get("monitor", "loss"))
mode = str(early_stopping.get("mode", "min")).lower()
enabled = bool(early_stopping.get("enabled", True))
log_interval = max(1, int(train_config.get("log_interval", 10)))
scalar_log_interval = max(1, int(train_config.get("scalar_log_interval", log_interval)))
# DataLoader 统一走同一个 collate_fn, 兼容每张图目标数量不同的情况。
train_loader_kwargs = {
    "batch_size": max(batch_size, 1),
    "shuffle": True,
    "num_workers": num_workers,
    "collate_fn": self._collate_batch,
    "pin_memory": pin_memory,
}
val_loader_kwargs = {
    "batch_size": max(eval_batch_size, 1),
    "shuffle": False,
    "num_workers": num_workers,
    "collate_fn": self._collate_batch,
    "pin_memory": pin_memory,
}
if num_workers > 0:
    train_loader_kwargs["persistent_workers"] = persistent_workers
    val_loader_kwargs["persistent_workers"] = persistent_workers
    train_loader_kwargs["prefetch_factor"] = prefetch_factor
    val_loader_kwargs["prefetch_factor"] = prefetch_factor
train_loader = DataLoader(
    train_dataset,
    **train_loader_kwargs,
)
val_loader = DataLoader(
    val_dataset,
    **val_loader_kwargs,
)
best_metric = float("inf") if mode == "min" else float("-inf")
epochs_without_improvement = 0
best_epoch = 0
final_val_metrics: dict[str, float] | None = None

self.logger.info(
    "[train] experiment=%s dataset_size=%s batch_size=%s epochs=%s max_batches=%s device=%s
early_stopping=%s monitor=%s mode=%s patience=%s min_delta=%.6f",
    self.config.get("experiment", {}).get("name", "unknown"),
    len(train_dataset),
    batch_size,
    epochs,
    max_batches,
    self.device,
    enabled,
    monitor,
    mode,
    patience,
    min_delta,
)
self.logger.info("[train] tensorboard_log_dir=%s", self.run_dir)

```

```

self.logger.info(
    "[train] dataloader workers=%s pin_memory=%s persistent_workers=%s prefetch_factor=%s log_interval=%s
    scalar_log_interval=%s amp=%s eval_interval=%s",
    num_workers,
    pin_memory,
    persistent_workers,
    prefetch_factor if num_workers > 0 else 0,
    log_interval,
    scalar_log_interval,
    use_amp,
    eval_interval,
)

try:
    self._write_config_snapshot()
    self._write_experiment_summary(loss_fn=loss_fn)
    self._log_hparams(tracker)
    self._prepare_dataset_cache(train_dataset, split="train")
    self._prepare_dataset_cache(val_dataset, split="val")
    if self.checkpoint_path.exists():
        try:
            # 若存在 latest checkpoint, 则优先从最近训练进度继续。
            metadata = model.load_checkpoint(
                self.checkpoint_path,
                optimizer=optimizer,
                map_location=self.device,
            )
            self.logger.info(
                "[train] resumed checkpoint=%s metadata=%s",
                self.checkpoint_path,
                metadata,
            )
        except RuntimeError as exc:
            self.logger.warning(
                "[train] skipped incompatible checkpoint=%s error=%s",
                self.checkpoint_path,
                exc,
            )

    global_step = 0
    for epoch_index in range(epochs):
        model.train()
        epoch_loss = 0.0
        epoch_box_iou = 0.0
        epoch_correct = 0
        sample_count = 0

        for batch_index, batch in enumerate(train_loader):
            if max_batches > 0 and batch_index >= max_batches:
                break

            raw_image = batch["raw_image"].to(self.device, non_blocking=pin_memory)
            enhanced_image = batch["enhanced_image"].to(self.device, non_blocking=pin_memory)
            if global_step == 0 and log_graph:
                self._try_log_graph(tracker, model, raw_image, enhanced_image)
            optimizer.zero_grad(set_to_none=True)
            with torch.amp.autocast(device_type=self.device.type, enabled=use_amp):
                predictions = model(raw_image, enhanced_image)
                # 先把固定 query 与真实目标对齐, 再进入损失计算。
                matched_classes, matched_boxes, matched_mask = self._match_batch(predictions, batch)
                loss_output = loss_fn(predictions, matched_classes, matched_boxes, matched_mask)
                loss = loss_output["loss"]
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

            batch_iou = self._mean_iou(loss_output["pred_boxes"], matched_boxes, matched_mask)
            batch_correct = self._count_class_matches(
                loss_output["predicted_class"],
                matched_classes,
                matched_mask,

```

```

)
batch_size_value = int(matched_mask.sum().item())
batch_loss_value = float(loss.item())
batch_acc_value = batch_correct / max(batch_size_value, 1)

# 训练统计只按有效匹配目标聚合, 避免背景 query 拉低可解释性。
epoch_loss += batch_loss_value * batch_size_value
epoch_box_iou += batch_iou * batch_size_value
epoch_correct += batch_correct
sample_count += batch_size_value
global_step += 1

if global_step % scalar_log_interval == 0:
    tracker.log_scalar("train/loss_step", batch_loss_value, global_step)
    tracker.log_scalar("train/lr", self._current_lr(optimizer), global_step)
    tracker.log_scalar("train/batch_acc", batch_acc_value, global_step)
    tracker.log_scalar("train/batch_box_iou", batch_iou, global_step)
    tracker.log_scalar(
        "train/batch_cls_loss",
        float(loss_output["classification_loss"].item()),
        global_step,
    )
    tracker.log_scalar(
        "train/batch_box_loss",
        float(loss_output["box_loss"].item()),
        global_step,
    )
if (batch_index + 1) % log_interval == 0 or batch_index == 0:
    self.logger.info(
        "[train] epoch=%s batch=%s loss=%.4f acc=%.2f iou=%.2f objects=%s",
        epoch_index + 1,
        batch_index + 1,
        batch_loss_value,
        batch_acc_value,
        batch_iou,
        batch_size_value,
    )

epoch_loss_value = epoch_loss / max(sample_count, 1)
epoch_acc_value = epoch_correct / max(sample_count, 1)
epoch_iou_value = epoch_box_iou / max(sample_count, 1)
tracker.log_scalar("train/loss_epoch", epoch_loss_value, epoch_index + 1)
tracker.log_scalar("train/epoch_acc", epoch_acc_value, epoch_index + 1)
tracker.log_scalar("train/epoch_box_iou", epoch_iou_value, epoch_index + 1)
if log_histograms:
    tracker.log_parameter_histograms(model.named_parameters(), epoch_index + 1)
# 每轮先保存 latest, 便于异常中断后继续训练。
model.save_checkpoint(
    self.checkpoint_path,
    optimizer=optimizer,
    metadata={
        "experiment": self.config.get("experiment", {}).get("name", "unknown"),
        "epoch": epoch_index + 1,
        "samples": sample_count,
        "device": str(self.device),
    },
)
)
should_eval = (epoch_index + 1) % eval_interval == 0 or (epoch_index + 1) == epochs
if not should_eval:
    self.logger.info(
        "[train] epoch=%s skipped validation eval_interval=%s",
        epoch_index + 1,
        eval_interval,
    )
    continue
val_metrics = self._evaluate_model(
    model=model,
    loss_fn=loss_fn,
    val_loader=val_loader,
    max_batches=eval_max_batches,
    conf_thresh=float(eval_config.get("conf_thresh", 0.25)),

```

```

        iou_thresh=float(eval_config.get("iou_thresh", 0.5)),
        tracker=tracker,
        tracker_step=epoch_index + 1,
        metric_prefix="val",
        use_amp=use_amp,
        pin_memory=pin_memory,
    )
    final_val_metrics = val_metrics
    monitored_value = float(val_metrics[monitor])
    tracker.log_scalar(f"train/early_stop_{monitor}", monitored_value, epoch_index + 1)
    tracker.flush()
    improved = self._is_improved(
        current=monitored_value,
        best=best_metric,
        mode=mode,
        min_delta=min_delta,
    )
    if improved:
        best_metric = monitored_value
        best_epoch = epoch_index + 1
        epochs_without_improvement = 0
        # 只有监控指标改善时才刷新 best checkpoint.
        model.save_checkpoint(
            self.best_checkpoint_path,
            optimizer=optimizer,
            metadata={
                "experiment": self.config.get("experiment", {}).get("name", "unknown"),
                "epoch": epoch_index + 1,
                "samples": sample_count,
                "device": str(self.device),
                "best_metric": best_metric,
                "monitor": monitor,
            },
        )
    else:
        epochs_without_improvement += 1
    self.logger.info(
        "[train] epoch=%s summary loss=%.4f acc=%.2f iou=%.2f val_%.4f best=%.4f best_epoch=%s"
        wait=%s/%s samples=%s",
        epoch_index + 1,
        epoch_loss_value,
        epoch_acc_value,
        epoch_iou_value,
        monitor,
        monitored_value,
        best_metric,
        best_epoch,
        epochs_without_improvement,
        patience,
        sample_count,
    )
    if enabled and epochs_without_improvement >= patience:
        self.logger.info(
            "[train] early stopping triggered at epoch=%s best_epoch=%s best_%.4f",
            epoch_index + 1,
            best_epoch,
            monitor,
            best_metric,
        )
        break
    if final_val_metrics is not None:
        self._log_hparams(tracker, final_val_metrics, prefix="val")
        self.last_fit_summary = {
            "best_epoch": best_epoch,
            "best_metric": best_metric,
            "monitor": monitor,
            "last_val_metrics": final_val_metrics,
            "run_dir": str(self.run_dir) if self.run_dir is not None else None,
            "checkpoint_path": str(self.checkpoint_path),
            "best_checkpoint_path": str(self.best_checkpoint_path),
        }
}

```

```

        return self.last_fit_summary
    finally:
        tracker.close()

def evaluate(self) -> dict[str, float]:
    """加载 checkpoint 并输出评估指标、复杂度和推理速度。"""
    _, val_dataset = self._build_datasets()
    model = self._build_model().to(self.device)
    loss_fn = DetectionLoss().to(self.device)
    tracker = ExperimentTracker(self.config)
    self.run_dir = tracker.run_dir
    use_amp = self.device.type == "cuda" and bool(self.config.get("train", {}).get("amp", True))
    batch_size = min(int(self.config.get("train", {}).get("batch_size", 4)), len(val_dataset))
    max_batches = int(self.config.get("eval", {}).get("max_batches", 2))
    self._prepare_dataset_cache(val_dataset, split="eval")
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=max(0, int(self.config.get("dataset", {}).get("workers", 0))),
        collate_fn=self._collate_batch,
        pin_memory=self.device.type == "cuda",
    )

    try:
        eval_checkpoint = self._resolve_eval_checkpoint()
        if eval_checkpoint is not None:
            try:
                # 默认优先评估 best checkpoint, 没有时回退到 latest。
                metadata = model.load_checkpoint(eval_checkpoint, map_location=self.device)
                self.logger.info("[eval] loaded checkpoint=%s metadata=%s", eval_checkpoint, metadata)
            except RuntimeError as exc:
                self.logger.warning(
                    "[eval] skipped incompatible checkpoint=%s error=%s",
                    eval_checkpoint,
                    exc,
                )
        else:
            self.logger.warning("[eval] checkpoint not found, using fresh model=%s", self.checkpoint_path)

        self.logger.info(
            "[eval] experiment=%s dataset_size=%s batch_size=%s max_batches=%s device=%s",
            self.config.get("experiment", {}).get("name", "unknown"),
            len(val_dataset),
            batch_size,
            max_batches,
            self.device,
        )

        self.logger.info("[eval] tensorboard_log_dir=%s", self.run_dir)

        model.eval()
        eval_config = self.config.get("eval", {})
        metrics = self._evaluate_model(
            model=model,
            loss_fn=loss_fn,
            val_loader=val_loader,
            max_batches=max_batches,
            conf_thresh=float(eval_config.get("conf_thresh", 0.25)),
            iou_thresh=float(eval_config.get("iou_thresh", 0.5)),
            tracker=tracker,
            tracker_step=1,
            metric_prefix="eval",
            use_amp=use_amp,
            pin_memory=self.device.type == "cuda",
        )

        # 额外补充参数量、GFLOPs 与 FPS, 便于和论文表格对齐。
        complexity_metrics = self._complexity_metrics(model, self._last_profile_batch)
        speed_metrics = self._speed_metrics(
            model=model,
            dataset=val_dataset,
            conf_thresh=float(eval_config.get("conf_thresh", 0.25)),

```

```

        iou_thresh=float(eval_config.get("iou_thresh", 0.5)),
        max_batches=min(max_batches, 10),
        batch_size=min(int(self.config.get("train", {}).get("batch_size", 4)), len(val_dataset)),
        use_amp=use_amp,
    )
    metrics.update(complexity_metrics)
    metrics.update(speed_metrics)
    tracker.log_scalar("eval/params_m", metrics["params_m"], 1)
    tracker.log_scalar("eval/gflops", metrics["gflops"], 1)
    tracker.log_scalar("eval/fps", metrics["fps"], 1)
    self._log_hparams(tracker, metrics, prefix="eval")
    self._write_eval_artifacts(self._last_predictions, metrics)
    self.logger.info(
        "[eval] summary loss=%4f acc=%2f iou=%2f map50=%3f map50_95=%3f params=%3fM gflops=%3f
        fps=%2f samples=%s",
        metrics["loss"],
        metrics["acc"],
        metrics["box_iou"],
        metrics["map50"],
        metrics["map50_95"],
        metrics["params_m"],
        metrics["gflops"],
        metrics["fps"],
        self._last_eval_samples,
    )
    return metrics
finally:
    tracker.close()

@property
def _background_class(self) -> int:
    """背景类别索引固定放在真实类别之后。"""
    return int(self.config.get("dataset", {}).get("num_classes", 4))

def _build_datasets(self) -> tuple[UnderwaterDetectionDataset, UnderwaterDetectionDataset]:
    """根据配置构建训练集和验证集。"""
    dataset_config = self.config.get("dataset", {})
    common_args = {
        "num_classes": int(dataset_config.get("num_classes", 4)),
        "image_size": int(dataset_config.get("image_size", 128)),
        "synthetic_size": int(dataset_config.get("synthetic_size", 16)),
        "max_objects": int(dataset_config.get("max_objects", 8)),
    }
    train_root = self._as_path(dataset_config.get("train_root"))
    val_root = self._as_path(dataset_config.get("val_root"))
    train_dataset = UnderwaterDetectionDataset(
        image_root=train_root,
        annotation_root=self._derive_annotation_root(train_root),
        enhanced_root=self._as_path(dataset_config.get("enhanced_root")),
        **common_args,
    )
    val_dataset = UnderwaterDetectionDataset(
        image_root=val_root,
        annotation_root=self._derive_annotation_root(val_root),
        enhanced_root=self._as_path(dataset_config.get("enhanced_root")),
        **common_args,
    )
    return train_dataset, val_dataset

def _build_model(self) -> torch.nn.Module:
    """按配置选择 QDCR-Net 或单分支基线模型。"""
    dataset_config = self.config.get("dataset", {})
    model_config = self.config.get("model", {})
    model_name = str(model_config.get("name", "qdcr_net")).lower()
    model_cls = BaselineDetector if "baseline" in model_name else QDCRNet
    common_kwargs = {
        "num_classes": int(dataset_config.get("num_classes", 4)),
        "feature_dim": int(model_config.get("feature_dim", 32)),
        "num_queries": int(model_config.get("num_queries", dataset_config.get("max_objects", 8))),
    }
    if model_cls is BaselineDetector:

```

```

        return model_cls(**common_kwargs)
    return model_cls(
        **common_kwargs,
        cross_residual_mode=str(model_config.get("cross_residual_mode", "vanilla")),
        cross_residual_threshold_init=float(model_config.get("cross_residual_threshold_init", 0.1)),
        cross_residual_threshold_slope=float(model_config.get("cross_residual_threshold_slope", 10.0)),
        cross_residual_sparse_mode=str(model_config.get("cross_residual_sparse_mode", "soft_threshold")),
        cross_residual_sparse_lambda_init=float(
            model_config.get("cross_residual_sparse_lambda_init", 0.1)
        ),
        cross_residual_topk_ratio=float(model_config.get("cross_residual_topk_ratio", 0.5)),
    )

def _build_optimizer(self, model: torch.nn.Module) -> torch.optim.Optimizer:
    """构建优化器，默认使用 AdamW。"""
    train_config = self.config.get("train", {})
    optimizer_name = str(train_config.get("optimizer", "adamw")).lower()
    learning_rate = float(train_config.get("lr", 1e-3))
    weight_decay = float(train_config.get("weight_decay", 0.0))
    if optimizer_name == "sgd":
        return torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9, weight_decay=weight_decay)
    return torch.optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=weight_decay)

def _derive_annotation_root(self, image_root: Path | None) -> Path | None:
    """从图片目录推断 labels 目录位置。"""
    if image_root is None:
        return None
    if image_root.name == "images":
        return image_root.parent / "labels"
    return image_root / "labels"

def _as_path(self, value: str | None) -> Path | None:
    """把配置中的可选字符串路径转换为 Path。"""
    return None if value is None else Path(value)

def _set_seed(self, seed: int) -> None:
    """设置 CPU/CUDA 随机种子，减少实验抖动。"""
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def _collate_batch(self, batch: list[dict]) -> dict[str, object]:
    """自定义拼批逻辑，保留变长目标框列表。"""
    return {
        "sample_id": [sample["sample_id"] for sample in batch],
        "raw_image": torch.stack([sample["raw_image"] for sample in batch], dim=0),
        "enhanced_image": torch.stack([sample["enhanced_image"] for sample in batch], dim=0),
        "gt_boxes": [sample["gt_boxes"] for sample in batch],
        "gt_classes": [sample["gt_classes"] for sample in batch],
        "source": [sample["source"] for sample in batch],
    }

def _match_batch(
    self,
    predictions: dict[str, torch.Tensor],
    batch: dict[str, object],
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """对一个 batch 中的每张图片分别执行贪心匹配。"""
    logits = predictions["logits"]
    pred_boxes = predictions["pred_boxes"]
    gt_classes_list: list[torch.Tensor] = batch["gt_classes"] # type: ignore[assignment]
    gt_boxes_list: list[torch.Tensor] = batch["gt_boxes"] # type: ignore[assignment]

    matched_classes_list = []
    matched_boxes_list = []
    matched_mask_list = []
    for batch_index in range(logits.size(0)):
        gt_classes = gt_classes_list[batch_index].to(self.device)
        gt_boxes = gt_boxes_list[batch_index].to(self.device)
        matched_classes, matched_boxes, matched_mask = greedy_match(
```

```

        logits[batch_index],
        pred_boxes[batch_index],
        gt_classes,
        gt_boxes,
        background_class=self._background_class,
    )
    matched_classes_list.append(matched_classes)
    matched_boxes_list.append(matched_boxes)
    matched_mask_list.append(matched_mask)

    return (
        torch.stack(matched_classes_list, dim=0),
        torch.stack(matched_boxes_list, dim=0),
        torch.stack(matched_mask_list, dim=0),
    )

def _mean_iou(
    self,
    pred_boxes: torch.Tensor,
    target_boxes: torch.Tensor,
    valid_mask: torch.Tensor,
) -> float:
    """只在有效匹配位置上计算平均 IoU。"""
    iou = pairwise_iou_xywh(
        pred_boxes[valid_mask],
        target_boxes[valid_mask],
    )
    if iou.numel() == 0:
        return 0.0
    diagonal = torch.diag(iou)
    return float(diagonal.mean().item()) if diagonal.numel() else 0.0

def _count_class_matches(
    self,
    predicted_class: torch.Tensor,
    targets: torch.Tensor,
    valid_mask: torch.Tensor,
) -> int:
    """统计有效匹配目标上的类别预测正确数。"""
    valid_predictions = predicted_class[valid_mask]
    valid_targets = targets[valid_mask]
    if valid_targets.numel() == 0:
        return 0
    return int((valid_predictions == valid_targets).sum().item())

def _ground_truth_records(self, batch: dict[str, object]) -> list[dict[str, torch.Tensor]]:
    """把 batch 中的 GT 整理成评估函数需要的记录格式。"""
    gt_classes_list: list[torch.Tensor] = batch["gt_classes"] # type: ignore[assignment]
    gt_boxes_list: list[torch.Tensor] = batch["gt_boxes"] # type: ignore[assignment]
    return [
        {
            "classes": gt_classes.cpu(),
            "boxes": gt_boxes.cpu(),
        }
        for gt_classes, gt_boxes in zip(gt_classes_list, gt_boxes_list)
    ]

def _write_eval_artifacts(
    self,
    predictions: list[dict[str, torch.Tensor]],
    metrics: dict[str, float],
) -> None:
    """将预测结果和指标写出为 JSON 文件。"""
    self.output_dir.mkdir(parents=True, exist_ok=True)
    serializable_predictions = []
    for image_index, prediction in enumerate(predictions):
        serializable_predictions.append(
            {
                "image_index": image_index,
                "scores": prediction["scores"].cpu().tolist(),
                "classes": prediction["classes"].cpu().tolist(),
            }
        )

```

```

        "boxes": prediction["boxes"].cpu().tolist(),
    }
)

self.prediction_path.write_text(
    json.dumps(serializable_predictions, indent=2),
    encoding="utf-8",
)
self.metrics_path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")

def _resolve_eval_checkpoint(self) -> Path | None:
    """根据评估配置决定优先使用 best 还是 latest checkpoint。"""
    eval_config = self.config.get("eval", {})
    checkpoint_mode = str(eval_config.get("checkpoint", "best")).lower()
    if checkpoint_mode == "latest":
        return self.checkpoint_path if self.checkpoint_path.exists() else None
    if self.best_checkpoint_path.exists():
        return self.best_checkpoint_path
    if self.checkpoint_path.exists():
        return self.checkpoint_path
    return None

def _evaluate_model(
    self,
    model: torch.nn.Module,
    loss_fn: DetectionLoss,
    val_loader: DataLoader,
    max_batches: int,
    conf_thresh: float,
    iou_thresh: float,
    tracker: ExperimentTracker | None = None,
    tracker_step: int | None = None,
    metric_prefix: str = "val",
    use_amp: bool = False,
    pin_memory: bool = False,
) -> dict[str, float]:
    """在验证集上运行模型并汇总核心指标。"""
    total_loss = 0.0
    total_iou = 0.0
    total_correct = 0
    sample_count = 0
    all_predictions: list[dict[str, torch.Tensor]] = []
    all_ground_truths: list[dict[str, torch.Tensor]] = []
    profile_batch: dict[str, torch.Tensor] | None = None

    model.eval()
    with torch.no_grad():
        for batch_index, batch in enumerate(val_loader):
            if max_batches > 0 and batch_index >= max_batches:
                break

            raw_image = batch["raw_image"].to(self.device, non_blocking=pin_memory)
            enhanced_image = batch["enhanced_image"].to(self.device, non_blocking=pin_memory)
            if profile_batch is None:
                # 保留首个 batch 的一个样本，用于后续复杂度估算。
                profile_batch = {
                    "raw_image": raw_image[:1],
                    "enhanced_image": enhanced_image[:1],
                }
            with torch.amp.autocast(device_type=self.device.type, enabled=use_amp):
                predictions = model(raw_image, enhanced_image)
                matched_classes, matched_boxes, matched_mask = self._match_batch(predictions, batch)
                loss_output = loss_fn(predictions, matched_classes, matched_boxes, matched_mask)

            batch_size_value = int(matched_mask.sum().item())
            total_loss += float(loss_output["loss"].item()) * batch_size_value
            total_iou += self._mean_iou(loss_output["pred_boxes"], matched_boxes, matched_mask) *
batch_size_value
            total_correct += self._count_class_matches(
                loss_output["predicted_class"],
                matched_classes,

```

```

        matched_mask,
    )
    sample_count += batch_size_value

    decoded = decode_predictions(
        predictions["logits"],
        predictions["pred_boxes"],
        background_class=self._background_class,
        conf_thresh=conf_thresh,
        iou_thresh=iou_thresh,
    )
    all_predictions.extend(decoded)
    all_ground_truths.extend(self._ground_truth_records(batch))

# 基础损失/精度指标和检测指标最后统一合并，便于日志与 JSON 复用。
metrics = {
    "loss": total_loss / max(sample_count, 1),
    "acc": total_correct / max(sample_count, 1),
    "box_iou": total_iou / max(sample_count, 1),
}
metrics.update(
    compute_detection_metrics(
        all_predictions,
        all_ground_truths,
        num_classes=int(self.config.get("dataset", {}).get("num_classes", 4)),
    )
)
self._last_predictions = all_predictions
self._last_ground_truths = all_ground_truths
self._last_profile_batch = profile_batch
self._last_eval_samples = sample_count
if tracker is not None and tracker_step is not None:
    tracker.log_scalar(f"{metric_prefix}/loss", metrics["loss"], tracker_step)
    tracker.log_scalar(f"{metric_prefix}/acc", metrics["acc"], tracker_step)
    tracker.log_scalar(f"{metric_prefix}/box_iou", metrics["box_iou"], tracker_step)
    tracker.log_scalar(f"{metric_prefix}/mAP", metrics["map50"], tracker_step)
    tracker.log_scalar(f"{metric_prefix}/mAP50_95", metrics["map50_95"], tracker_step)
    tracker.log_scalar(f"{metric_prefix}/precision", metrics["precision"], tracker_step)
    tracker.log_scalar(f"{metric_prefix}/recall", metrics["recall"], tracker_step)
return metrics

def _log_hparams(
    self,
    tracker: ExperimentTracker,
    metrics: dict[str, float] | None = None,
    prefix: str = "hparams",
) -> None:
    """把关键配置写入 TensorBoard HParams 面板。"""
    payload = self._hparams_payload()
    if metrics is None:
        tracker.log_hparams(payload)
        return
    summary_metrics = {
        f"{prefix}/loss": float(metrics.get("loss", 0.0)),
        f"{prefix}/acc": float(metrics.get("acc", 0.0)),
    }
    if "map50" in metrics:
        summary_metrics[f"{prefix}/mAP"] = float(metrics["map50"])
    if "precision" in metrics:
        summary_metrics[f"{prefix}/precision"] = float(metrics["precision"])
    if "recall" in metrics:
        summary_metrics[f"{prefix}/recall"] = float(metrics["recall"])
    tracker.log_hparams(payload, summary_metrics)

def _hparams_payload(self) -> dict[str, Any]:
    experiment_config = self.config.get("experiment", {})
    dataset_config = self.config.get("dataset", {})
    model_config = self.config.get("model", {})
    train_config = self.config.get("train", {})
    eval_config = self.config.get("eval", {})
    early_stopping = train_config.get("early_stopping", {})

```

```

return {
    "experiment": experiment_config.get("name", "unknown"),
    "model": model_config.get("name", "unknown"),
    "dataset": self._dataset_name(),
    "batch_size": int(train_config.get("batch_size", 4)),
    "learning_rate": float(train_config.get("lr", 1e-3)),
    "epochs": int(train_config.get("epochs", 1)),
    "optimizer": train_config.get("optimizer", "adamw"),
    "scheduler": train_config.get("scheduler", "none"),
    "seed": int(experiment_config.get("seed", 42)),
    "device": str(self.device),
    "weight_decay": float(train_config.get("weight_decay", 0.0)),
    "image_size": int(dataset_config.get("image_size", 128)),
    "num_workers": int(dataset_config.get("workers", 0)),
    "pin_memory": self.device.type == "cuda",
    "persistent_workers": int(dataset_config.get("workers", 0)) > 0,
    "prefetch_factor": int(dataset_config.get("prefetch_factor", 2)),
    "cache_preprocessed": bool(dataset_config.get("cache_preprocessed", True)),
    "feature_dim": int(model_config.get("feature_dim", 32)),
    "num_queries": int(model_config.get("num_queries", dataset_config.get("max_objects", 8))),
    "cross_residual_mode": str(model_config.get("cross_residual_mode", "vanilla")),
    "cross_residual_threshold_init": float(model_config.get("cross_residual_threshold_init", 0.1)),
    "cross_residual_threshold_slope": float(model_config.get("cross_residual_threshold_slope", 10.0)),
    "cross_residual_sparse_mode": str(model_config.get("cross_residual_sparse_mode", "soft_threshold")),
    "cross_residual_sparse_lambda_init": float(
        model_config.get("cross_residual_sparse_lambda_init", 0.1)
    ),
    "cross_residual_topk_ratio": float(model_config.get("cross_residual_topk_ratio", 0.5)),
    "conf_thresh": float(eval_config.get("conf_thresh", 0.25)),
    "iou_thresh": float(eval_config.get("iou_thresh", 0.5)),
    "early_stop_monitor": early_stopping.get("monitor", "loss"),
    "log_interval": int(train_config.get("log_interval", 10)),
    "scalar_log_interval": int(train_config.get("scalar_log_interval", train_config.get("log_interval",
10))),
    "eval_interval": int(train_config.get("eval_interval", 5)),
    "amp": bool(train_config.get("amp", True)),
    "log_graph": bool(train_config.get("log_graph", False)),
    "log_histograms": bool(train_config.get("log_histograms", False)),
    "loss_box_weight": 5.0,
    "loss_iou_weight": 2.0,
    "loss_focal_gamma": 2.0,
    "loss_background_weight": 0.2,
}

def _write_config_snapshot(self) -> None:
    """把完整实验配置复制到输出目录，便于事后复现实验。"""
    self.output_dir.mkdir(parents=True, exist_ok=True)
    self.config_snapshot_path.write_text(
        yaml.safe_dump(self.config, sort_keys=False, allow_unicode=True),
        encoding="utf-8",
    )

def _write_experiment_summary(self, loss_fn: DetectionLoss) -> None:
    """额外写出一份关键超参摘要，减少后续查日志成本。"""
    experiment_config = self.config.get("experiment", {})
    dataset_config = self.config.get("dataset", {})
    model_config = self.config.get("model", {})
    train_config = self.config.get("train", {})
    eval_config = self.config.get("eval", {})
    early_stopping = train_config.get("early_stopping", {})
    summary = {
        "experiment_name": experiment_config.get("name", "unknown"),
        "output_dir": str(self.output_dir),
        "runs_dir": experiment_config.get("runs_dir", ""),
        "device": str(self.device),
        "seed": int(experiment_config.get("seed", 42)),
        "dataset": {
            "name": self._dataset_name(),
            "train_root": dataset_config.get("train_root", ""),
            "val_root": dataset_config.get("val_root", ""),
            "enhanced_root": dataset_config.get("enhanced_root", ""),

```

```

        "num_classes": int(dataset_config.get("num_classes", 4)),
        "image_size": int(dataset_config.get("image_size", 128)),
        "max_objects": int(dataset_config.get("max_objects", 8)),
        "workers": int(dataset_config.get("workers", 0)),
        "prefetch_factor": int(dataset_config.get("prefetch_factor", 2)),
        "cache_preprocessed": bool(dataset_config.get("cache_preprocessed", True)),
        "cache_dir": str(dataset_config.get("cache_dir", "")),
    },
    "model": {
        "name": model_config.get("name", "unknown"),
        "feature_dim": int(model_config.get("feature_dim", 32)),
        "num_queries": int(model_config.get("num_queries", dataset_config.get("max_objects", 8))),
        "quality_aware": bool(model_config.get("quality_aware", False)),
        "cross_residual_mode": str(model_config.get("cross_residual_mode", "vanilla")),
        "cross_residual_threshold_init": float(model_config.get("cross_residual_threshold_init", 0.1)),
        "cross_residual_threshold_slope": float(model_config.get("cross_residual_threshold_slope", 10.0)),
        "cross_residual_sparse_mode": str(model_config.get("cross_residual_sparse_mode",
"soft_threshold")),
        "cross_residual_sparse_lambda_init": float(
            model_config.get("cross_residual_sparse_lambda_init", 0.1)
        ),
        "cross_residual_topk_ratio": float(model_config.get("cross_residual_topk_ratio", 0.5)),
        "cross_residual_stages": model_config.get("cross_residual_stages", []),
        "small_object_neck": bool(model_config.get("small_object_neck", False)),
    },
    "train": {
        "epochs": int(train_config.get("epochs", 1)),
        "batch_size": int(train_config.get("batch_size", 4)),
        "optimizer": train_config.get("optimizer", "adamw"),
        "learning_rate": float(train_config.get("lr", 1e-3)),
        "weight_decay": float(train_config.get("weight_decay", 0.0)),
        "max_batches_per_epoch": int(train_config.get("max_batches_per_epoch", 0)),
        "log_interval": int(train_config.get("log_interval", 10)),
        "scalar_log_interval": int(
            train_config.get("scalar_log_interval", train_config.get("log_interval", 10))
        ),
        "eval_interval": int(train_config.get("eval_interval", 5)),
        "amp": bool(train_config.get("amp", True)),
        "log_graph": bool(train_config.get("log_graph", False)),
        "log_histograms": bool(train_config.get("log_histograms", False)),
        "pin_memory": self.device.type == "cuda",
        "persistent_workers": int(dataset_config.get("workers", 0)) > 0,
        "early_stopping": {
            "enabled": bool(early_stopping.get("enabled", True)),
            "monitor": early_stopping.get("monitor", "loss"),
            "mode": early_stopping.get("mode", "min"),
            "patience": int(early_stopping.get("patience", 5)),
            "min_delta": float(early_stopping.get("min_delta", 1e-4)),
        },
    },
    "eval": {
        "checkpoint": eval_config.get("checkpoint", "best"),
        "max_batches": int(eval_config.get("max_batches", 0)),
        "conf_thresh": float(eval_config.get("conf_thresh", 0.25)),
        "iou_thresh": float(eval_config.get("iou_thresh", 0.5)),
    },
    "loss": {
        "name": loss_fn.__class__.__name__,
        "box_weight": float(loss_fn.box_weight),
        "iou_weight": float(loss_fn.iou_weight),
        "focal_gamma": float(loss_fn.focal_gamma),
        "background_weight": float(loss_fn.background_weight),
        "box_regression": "SmoothL1Loss",
    },
}
self.summary_path.write_text(json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8")

def _dataset_name(self) -> str:
    dataset_config = self.config.get("dataset", {})
    dataset_name = dataset_config.get("name")
    if dataset_name:

```

```

        return str(dataset_name)
    for key in ("train_root", "val_root"):
        path = dataset_config.get(key)
        if not path:
            continue
        parts = Path(path).parts
        for marker in ("train", "valid", "val"):
            if marker in parts:
                index = parts.index(marker)
                if index > 0:
                    return parts[index - 1]
        if len(parts) >= 2:
            return parts[-2]
    return "dataset"

def _current_lr(self, optimizer: torch.optim.Optimizer) -> float:
    """读取当前优化器学习率。"""
    if not optimizer.param_groups:
        return 0.0
    return float(optimizer.param_groups[0].get("lr", 0.0))

def _try_log_graph(
    self,
    tracker: ExperimentTracker,
    model: torch.nn.Module,
    raw_image: torch.Tensor,
    enhanced_image: torch.Tensor,
) -> None:
    """在 TensorBoard 支持且图可追踪时记录模型结构。"""
    try:
        tracker.log_model_graph(model, (raw_image[:1], enhanced_image[:1]))
    except Exception as exc:
        self.logger.warning("[train] skipped tensorboard graph export error=%s", exc)

def _is_improved(self, current: float, best: float, mode: str, min_delta: float) -> bool:
    """判断当前监控指标是否达到“有效改善”。"""
    if mode == "max":
        return current > best + min_delta
    return current < best - min_delta

def _complexity_metrics(
    self,
    model: torch.nn.Module,
    profile_batch: dict[str, torch.Tensor] | None,
) -> dict[str, float]:
    """统计参数量，并在可行时估算 GFLOPs。"""
    params = float(sum(parameter.numel() for parameter in model.parameters()))
    gflops = 0.0
    if profile_batch is not None:
        gflops = self._estimate_gflops(
            model,
            profile_batch["raw_image"],
            profile_batch["enhanced_image"],
        )
    return {
        "params_m": params / 1_000_000.0,
        "gflops": gflops,
    }

def _estimate_gflops(
    self,
    model: torch.nn.Module,
    raw_image: torch.Tensor,
    enhanced_image: torch.Tensor,
) -> float:
    """通过 forward hook 粗略估算卷积和线性层 FLOPs。"""
    total_flops = 0.0
    handles: list[torch.utils.hooks.RemovableHandle] = []

    def conv_hook(module: nn.Conv2d, inputs: tuple[torch.Tensor, ...], output: torch.Tensor) -> None:
        nonlocal total_flops

```

```

        input_tensor = inputs[0]
        batch_size = float(input_tensor.shape[0])
        out_height = float(output.shape[2])
        out_width = float(output.shape[3])
        kernel_ops = float(module.kernel_size[0] * module.kernel_size[1] * (module.in_channels /
module.groups))
        output_elements = batch_size * float(module.out_channels) * out_height * out_width
        total_flops += output_elements * kernel_ops * 2.0

def linear_hook(module: nn.Linear, inputs: tuple[torch.Tensor, ...], output: torch.Tensor) -> None:
    nonlocal total_flops
    input_tensor = inputs[0]
    batch_elements = float(input_tensor.numel() / max(module.in_features, 1))
    total_flops += batch_elements * float(module.in_features) * float(module.out_features) * 2.0

for submodule in model.modules():
    if isinstance(submodule, nn.Conv2d):
        handles.append(submodule.register_forward_hook(conv_hook))
    elif isinstance(submodule, nn.Linear):
        handles.append(submodule.register_forward_hook(linear_hook))

was_training = model.training
model.eval()
with torch.no_grad():
    model(raw_image, enhanced_image)
if was_training:
    model.train()

for handle in handles:
    handle.remove()

return total_flops / 1_000_000_000.0

def _speed_metrics(
    self,
    model: torch.nn.Module,
    dataset: UnderwaterDetectionDataset,
    conf_thresh: float,
    iou_thresh: float,
    max_batches: int,
    batch_size: int,
    use_amp: bool,
) -> dict[str, float]:
    """估算推理 FPS，并跳过首个 batch 作为预热。"""
    if len(dataset) == 0 or max_batches <= 0:
        return {"fps": 0.0}

    loader = DataLoader(
        dataset,
        batch_size=max(batch_size, 1),
        shuffle=False,
        num_workers=0,
        collate_fn=self._collate_batch,
    )

    was_training = model.training
    model.eval()
    timed_images = 0
    timed_batches = 0
    start_time = 0.0

    with torch.no_grad():
        for batch_index, batch in enumerate(loader):
            if max_batches > 0 and batch_index >= max_batches:
                break
            raw_image = batch["raw_image"].to(self.device, non_blocking=self.device.type == "cuda")
            enhanced_image = batch["enhanced_image"].to(self.device, non_blocking=self.device.type == "cuda")
            if self.device.type == "cuda":
                torch.cuda.synchronize(self.device)
            batch_start = time.perf_counter()
            with torch.amp.autocast(device_type=self.device.type, enabled=use_amp):

```

```

        predictions = model(raw_image, enhanced_image)
        decode_predictions(
            predictions["logits"],
            predictions["pred_boxes"],
            background_class=self._background_class,
            conf_thresh=conf_thresh,
            iou_thresh=iou_thresh,
        )
    if self.device.type == "cuda":
        torch.cuda.synchronize(self.device)
    batch_elapsed = time.perf_counter() - batch_start
    if batch_index == 0:
        # 首个 batch 常常包含额外的初始化开销，不纳入速度统计。
        continue
    start_time += batch_elapsed
    timed_images += int(raw_image.size(0))
    timed_batches += 1

    if was_training:
        model.train()

    if timed_batches == 0 or start_time <= 0.0:
        return {"fps": 0.0}
    return {"fps": timed_images / start_time}

def _prepare_dataset_cache(self, dataset: UnderwaterDetectionDataset, split: str) -> None:
    """把图像预处理结果写入临时目录，减少训练期的重复 CPU 变换。"""
    dataset_config = self.config.get("dataset", {})
    cache_enabled = bool(dataset_config.get("cache_preprocessed", True))
    if not cache_enabled:
        return
    cache_root = dataset_config.get("cache_dir")
    if cache_root is None:
        cache_root = Path(tempfile.gettempdir()) / "qdcr_preprocessed"
    else:
        cache_root = Path(cache_root)
    dataset.prepare_cache(cache_root / split)

```

src/external/__init__.py

```

# -*- coding: utf-8 -*-
"""外部检测器适配层。"""

from .faster_rcnn_runner import FasterRCNNRunner
from .ultralytics_runner import UltralyticsRunner

__all__ = ["UltralyticsRunner", "FasterRCNNRunner"]

```

src/external/faster_rcnn_runner.py

```

# -*- coding: utf-8 -*-
"""Torchvision Faster R-CNN 训练/评估适配。"""

from __future__ import annotations

from dataclasses import dataclass
import json
from pathlib import Path
from types import SimpleNamespace
from typing import Any

import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

from src.datasets import UnderwaterDetectionDataset

```

```
from src.engine.detection_ops import compute_detection_metrics
from src.utils import TensorBoardLogger, get_logger
```

```
class TorchvisionDetectionDataset(Dataset):
```

```
    """把当前数据集包装成 torchvision 检测模型需要的格式。"""
```

```
    def __init__(self, dataset: UnderwaterDetectionDataset) -> None:
```

```
        self.dataset = dataset
```

```
        self.image_size = dataset.image_size
```

```
    def __len__(self) -> int:
```

```
        return len(self.dataset)
```

```
    def __getitem__(self, index: int) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

```
        sample = self.dataset[index]
```

```
        boxes = sample["gt_boxes"].clone()
```

```
        boxes_xyxy = torch.empty_like(boxes)
```

```
        boxes_xyxy[:, 0] = (boxes[:, 0] - boxes[:, 2] / 2.0) * self.image_size
```

```
        boxes_xyxy[:, 1] = (boxes[:, 1] - boxes[:, 3] / 2.0) * self.image_size
```

```
        boxes_xyxy[:, 2] = (boxes[:, 0] + boxes[:, 2] / 2.0) * self.image_size
```

```
        boxes_xyxy[:, 3] = (boxes[:, 1] + boxes[:, 3] / 2.0) * self.image_size
```

```
        target = {
```

```
            "boxes": boxes_xyxy,
```

```
            "labels": sample["gt_classes"] + 1,
```

```
            "image_id": torch.tensor([index], dtype=torch.long),
```

```
        }
```

```
        return sample["raw_image"], target
```

```
def _collate(batch: list[tuple[torch.Tensor, dict[str, torch.Tensor]]]) -> tuple[list[torch.Tensor], list[dict[str, torch.Tensor]]]:
```

```
    images, targets = zip(*batch)
```

```
    return list(images), list(targets)
```

```
@dataclass
```

```
class FasterRCNNRunner:
```

```
    """最小 Faster R-CNN 训练/评估控制器。"""
```

```
    config: dict[str, Any]
```

```
    def __post_init__(self) -> None:
```

```
        self.logger = get_logger("qdcf.faster_rcnn")
```

```
        self.experiment_config = self.config.get("experiment", {})
```

```
        self.dataset_config = self.config.get("dataset", {})
```

```
        self.train_config = self.config.get("train", {})
```

```
        self.eval_config = self.config.get("eval", {})
```

```
        self.output_dir = Path(self.experiment_config["output_dir"])
```

```
        self.output_dir.mkdir(parents=True, exist_ok=True)
```

```
        self.checkpoint_path = self.output_dir / "latest.pt"
```

```
        self.best_checkpoint_path = self.output_dir / "best.pt"
```

```
        self.metrics_path = self.output_dir / "metrics.json"
```

```
        self.device = torch.device(self.experiment_config.get("device", "cuda" if torch.cuda.is_available() else "cpu"))
```

```
        self.run_dir = TensorBoardLogger(self.config).run_dir
```

```
    def train(self) -> dict[str, float]:
```

```
        train_loader, val_loader = self._build_loaders()
```

```
        model = self._build_model().to(self.device)
```

```
        optimizer = torch.optim.SGD(model.parameters(), lr=float(self.train_config.get("lr", 0.005)), momentum=0.9, weight_decay=float(self.train_config.get("weight_decay", 0.0005)))
```

```
        best_loss = float("inf")
```

```
        best_metrics: dict[str, float] = {}
```

```
        epochs = int(self.train_config.get("epochs", 20))
```

```
        max_batches = int(self.train_config.get("max_batches_per_epoch", 0))
```

```
        for epoch in range(epochs):
```

```
            model.train()
```

```
            for batch_index, (images, targets) in enumerate(train_loader):
```

```
                if max_batches > 0 and batch_index >= max_batches:
```

```
                    break
```

```

        images = [image.to(self.device) for image in images]
        targets = [{key: value.to(self.device) for key, value in target.items()} for target in targets]
        loss_dict = model(images, targets)
        loss = sum(loss_dict.values())
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
        metrics = self._evaluate_model(model, val_loader)
        torch.save({"model_state": model.state_dict()}, self.checkpoint_path)
        if metrics["loss"] < best_loss:
            best_loss = metrics["loss"]
            best_metrics = metrics
            torch.save({"model_state": model.state_dict()}, self.best_checkpoint_path)
        self.logger.info("[faster_rcnn.train] epoch=%s val_loss=%.4f map50=%.4f", epoch + 1, metrics["loss"],
metrics["map50"])
        self.metrics_path.write_text(json.dumps(best_metrics, indent=2), encoding="utf-8")
        return best_metrics

def evaluate(self) -> dict[str, float]:
    _, val_loader = self._build_loaders()
    model = self._build_model().to(self.device)
    checkpoint = self.best_checkpoint_path if self.best_checkpoint_path.exists() else self.checkpoint_path
    if checkpoint.exists():
        payload = torch.load(checkpoint, map_location=self.device)
        model.load_state_dict(payload["model_state"])
    metrics = self._evaluate_model(model, val_loader)
    self.metrics_path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
    return metrics

def _build_model(self) -> nn.Module:
    num_classes = int(self.dataset_config.get("num_classes", 4)) + 1
    model = fasterrcnn_resnet50_fpn(weights=None, weights_backbone=None)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model

def _build_loaders(self) -> tuple[DataLoader, DataLoader]:
    common_args = {
        "num_classes": int(self.dataset_config.get("num_classes", 4)),
        "image_size": int(self.dataset_config.get("image_size", 320)),
        "synthetic_size": int(self.dataset_config.get("synthetic_size", 0)),
        "max_objects": int(self.dataset_config.get("max_objects", 8)),
    }
    train_dataset = UnderwaterDetectionDataset(
        image_root=Path(self.dataset_config["train_root"]),
        annotation_root=Path(self.dataset_config["train_root"]).parent.parent / "labels",
        enhanced_root=Path(self.dataset_config.get("enhanced_root", self.dataset_config["train_root"])),
        **common_args,
    )
    val_dataset = UnderwaterDetectionDataset(
        image_root=Path(self.dataset_config["val_root"]),
        annotation_root=Path(self.dataset_config["val_root"]).parent.parent / "labels",
        enhanced_root=Path(self.dataset_config.get("enhanced_root", self.dataset_config["train_root"])),
        **common_args,
    )
    train_loader = DataLoader(TorchvisionDetectionDataset(train_dataset),
batch_size=int(self.train_config.get("batch_size", 2)), shuffle=True, num_workers=0, collate_fn=_collate)
    val_loader = DataLoader(TorchvisionDetectionDataset(val_dataset),
batch_size=int(self.train_config.get("batch_size", 2)), shuffle=False, num_workers=0, collate_fn=_collate)
    return train_loader, val_loader

def _evaluate_model(self, model: nn.Module, val_loader: DataLoader) -> dict[str, float]:
    model.eval()
    predictions: list[dict[str, torch.Tensor]] = []
    ground_truths: list[dict[str, torch.Tensor]] = []
    total_loss = 0.0
    total_batches = 0
    with torch.no_grad():
        for images, targets in val_loader:
            images = [image.to(self.device) for image in images]
            eval_targets = [{key: value.to(self.device) for key, value in target.items()} for target in

```

```

targets]
    model.train()
    loss_dict = model(images, eval_targets)
    total_loss += float(sum(loss_dict.values()).item())
    total_batches += 1
    model.eval()
    outputs = model(images)
    for output, target in zip(outputs, targets):
        boxes = output["boxes"].detach().cpu().clone()
        boxes[:, 0::2] /= int(self.dataset_config.get("image_size", 320))
        boxes[:, 1::2] /= int(self.dataset_config.get("image_size", 320))
        boxes_xywh = torch.empty_like(boxes)
        boxes_xywh[:, 0] = (boxes[:, 0] + boxes[:, 2]) / 2.0
        boxes_xywh[:, 1] = (boxes[:, 1] + boxes[:, 3]) / 2.0
        boxes_xywh[:, 2] = boxes[:, 2] - boxes[:, 0]
        boxes_xywh[:, 3] = boxes[:, 3] - boxes[:, 1]
        predictions.append(
            {
                "scores": output["scores"].detach().cpu(),
                "classes": (output["labels"].detach().cpu() - 1).clamp(min=0),
                "boxes": boxes_xywh,
            }
        )
        gt_boxes_xyxy = target["boxes"].detach().cpu().clone()
        gt_boxes_xyxy[:, 0::2] /= int(self.dataset_config.get("image_size", 320))
        gt_boxes_xyxy[:, 1::2] /= int(self.dataset_config.get("image_size", 320))
        gt_boxes_xywh = torch.empty_like(gt_boxes_xyxy)
        gt_boxes_xywh[:, 0] = (gt_boxes_xyxy[:, 0] + gt_boxes_xyxy[:, 2]) / 2.0
        gt_boxes_xywh[:, 1] = (gt_boxes_xyxy[:, 1] + gt_boxes_xyxy[:, 3]) / 2.0
        gt_boxes_xywh[:, 2] = gt_boxes_xyxy[:, 2] - gt_boxes_xyxy[:, 0]
        gt_boxes_xywh[:, 3] = gt_boxes_xyxy[:, 3] - gt_boxes_xyxy[:, 1]
        ground_truths.append({"classes": target["labels"].detach().cpu() - 1, "boxes": gt_boxes_xywh})
    metrics = compute_detection_metrics(predictions, ground_truths, int(self.dataset_config.get("num_classes",
4)))
    metrics.update(
        {
            "loss": total_loss / max(total_batches, 1),
            "acc": 0.0,
            "box_iou": 0.0,
            "params_m": float(sum(p.numel() for p in model.parameters())) / 1_000_000.0,
            "gflops": 0.0,
            "fps": 0.0,
        }
    )
    return metrics

def notify_target(self) -> Any:
    return SimpleNamespace(run_dir=self.run_dir, best_checkpoint_path=self.best_checkpoint_path)

```

src/external/ultralytics_runner.py

```

# -*- coding: utf-8 -*-
"""Ultralytics 模型适配: YOLOv8 / RT-DETR / 增强后检测。"""

from __future__ import annotations

from dataclasses import dataclass
import json
from pathlib import Path
from types import SimpleNamespace
from typing import Any

import cv2
import yaml
from ultralytics import RTDETR, YOLO

from src.utils import TensorBoardLogger, get_logger

```

```

def _dataset_root_from_image_root(image_root: str | Path) -> Path:
    image_path = Path(image_root)
    if image_path.name == "images":
        return image_path.parent.parent
    return image_path.parent

@dataclass
class UltralyticsRunner:
    """对 Ultralytics 训练/评估接口做项目内封装。"""

    config: dict[str, Any]

    def __post_init__(self) -> None:
        self.logger = get_logger("qdc.ultralytics")
        self.experiment_config = self.config.get("experiment", {})
        self.dataset_config = self.config.get("dataset", {})
        self.model_config = self.config.get("model", {})
        self.train_config = self.config.get("train", {})
        self.eval_config = self.config.get("eval", {})
        self.output_dir = Path(self.experiment_config["output_dir"])
        self.output_dir.mkdir(parents=True, exist_ok=True)
        self.run_dir = TensorBoardLogger(self.config).run_dir
        self.best_checkpoint_path = self.output_dir / "weights" / "best.pt"
        self.metrics_path = self.output_dir / "metrics.json"
        self.config_snapshot_path = self.output_dir / "experiment_config.yaml"
        self.summary_path = self.output_dir / "experiment_summary.json"
        self.data_yaml_path = self._prepare_data_yaml()

    def train(self) -> dict[str, float]:
        self._write_config_snapshot()
        self._write_experiment_summary()
        model = self._build_model()
        self.logger.info("[ultralytics.train] model=%s data=%s output=%s", self.model_config["name"],
self.data_yaml_path, self.output_dir)
        model.train(
            data=str(self.data_yaml_path),
            epochs=int(self.train_config.get("epochs", 100)),
            batch=int(self.train_config.get("batch_size", 4)),
            imgsz=int(self.dataset_config.get("image_size", 640)),
            project=str(self.output_dir.parent),
            name=self.output_dir.name,
            exist_ok=True,
            device=str(self.experiment_config.get("device", "cuda")),
            workers=int(self.dataset_config.get("workers", 4)),
            patience=int(self.train_config.get("early_stopping", {}).get("patience", 10)),
            lr0=float(self.train_config.get("lr", 1e-3)),
        )
        return self.evaluate()

    def evaluate(self) -> dict[str, float]:
        model = self._build_model(weights_path=self._resolve_weights_path())
        split = str(self.eval_config.get("split", "val"))
        results = model.val(
            data=str(self.data_yaml_path),
            split=split,
            batch=int(self.train_config.get("batch_size", 4)),
            imgsz=int(self.dataset_config.get("image_size", 640)),
            device=str(self.experiment_config.get("device", "cuda")),
            conf=float(self.eval_config.get("conf_thresh", 0.25)),
            iou=float(self.eval_config.get("iou_thresh", 0.5)),
            project=str(self.output_dir.parent),
            name=f"{self.output_dir.name}_{split}_eval",
            exist_ok=True,
        )
        metrics = self._extract_metrics(results)
        self.metrics_path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
        return metrics

    def _write_config_snapshot(self) -> None:
        self.config_snapshot_path.write_text(

```

```

        yaml.safe_dump(self.config, sort_keys=False, allow_unicode=True),
        encoding="utf-8",
    )

def _write_experiment_summary(self) -> None:
    early_stopping = self.train_config.get("early_stopping", {})
    summary = {
        "experiment_name": self.experiment_config.get("name", "unknown"),
        "output_dir": str(self.output_dir),
        "runs_dir": self.experiment_config.get("runs_dir", ""),
        "device": str(self.experiment_config.get("device", "cuda")),
        "seed": int(self.experiment_config.get("seed", 42)),
        "dataset": {
            "name": self.dataset_config.get("name", "dataset"),
            "train_root": self.dataset_config.get("train_root", ""),
            "val_root": self.dataset_config.get("val_root", ""),
            "enhanced_root": self.dataset_config.get("enhanced_root", ""),
            "num_classes": int(self.dataset_config.get("num_classes", 0)),
            "image_size": int(self.dataset_config.get("image_size", 640)),
            "max_objects": int(self.dataset_config.get("max_objects", 0)),
        },
        "model": {
            "name": self.model_config.get("name", "unknown"),
            "weights": self.model_config.get("weights", ""),
        },
        "train": {
            "epochs": int(self.train_config.get("epochs", 100)),
            "batch_size": int(self.train_config.get("batch_size", 4)),
            "optimizer": "ultralytics_default",
            "learning_rate": float(self.train_config.get("lr", 1e-3)),
            "weight_decay": float(self.train_config.get("weight_decay", 0.0)),
            "early_stopping": {
                "patience": int(early_stopping.get("patience", 10)),
            },
        },
        "eval": {
            "split": self.eval_config.get("split", "val"),
            "conf_thresh": float(self.eval_config.get("conf_thresh", 0.25)),
            "iou_thresh": float(self.eval_config.get("iou_thresh", 0.5)),
        },
        "loss": {
            "name": "UltralyticsDetectionLoss",
            "components": ["box_loss", "cls_loss", "dfl_loss"],
        },
        "artifacts": {
            "data_yaml": str(self.data_yaml_path),
            "best_checkpoint": str(self.best_checkpoint_path),
            "metrics_path": str(self.metrics_path),
        },
    }
    self.summary_path.write_text(json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8")

def _build_model(self, weights_path: Path | None = None) -> Any:
    model_name = str(self.model_config.get("name", "yolov8"))
    weights = str(weights_path or self.model_config.get("weights", "yolov8n.pt"))
    if "rtdetr" in model_name.lower():
        return RTDETR(weights)
    return YOLO(weights)

def _resolve_weights_path(self) -> Path | None:
    if self.best_checkpoint_path.exists():
        return self.best_checkpoint_path
    return None

def _prepare_data_yaml(self) -> Path:
    if self.model_config.get("name") == "underwater_enhance_yolov8":
        return self._prepare_enhanced_dataset_yaml()

    dataset_root = _dataset_root_from_image_root(self.dataset_config["train_root"])
    data_yaml = dataset_root / "data.yaml"
    if data_yaml.exists():

```

```

        return data_yaml

generated_path = self.output_dir / "data.yaml"
payload = {
    "path": str(dataset_root.resolve()),
    "train": str(Path(self.dataset_config["train_root"]).resolve()),
    "val": str(Path(self.dataset_config["val_root"]).resolve()),
    "nc": int(self.dataset_config["num_classes"]),
    "names": self.dataset_config["class_names"],
}
generated_path.write_text(yaml.safe_dump(payload, sort_keys=False), encoding="utf-8")
return generated_path

def _prepare_enhanced_dataset_yaml(self) -> Path:
    dataset_root = _dataset_root_from_image_root(self.dataset_config["train_root"])
    cache_root = self.output_dir / "enhanced_dataset"
    for split in ("train", "valid", "test"):
        src_split = dataset_root / split
        if not src_split.exists():
            continue
        image_dir = src_split / "images"
        label_dir = src_split / "labels"
        out_image_dir = cache_root / split / "images"
        out_label_dir = cache_root / split / "labels"
        out_image_dir.mkdir(parents=True, exist_ok=True)
        out_label_dir.mkdir(parents=True, exist_ok=True)
        for image_path in sorted(image_dir.iterdir()):
            if not image_path.is_file():
                continue
            out_image_path = out_image_dir / image_path.name
            if not out_image_path.exists():
                image = cv2.imread(str(image_path))
                lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
                l, a, b = cv2.split(lab)
                clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
                enhanced_l = clahe.apply(l)
                merged = cv2.merge((enhanced_l, a, b))
                enhanced = cv2.cvtColor(merged, cv2.COLOR_LAB2BGR)
                cv2.imwrite(str(out_image_path), enhanced)
            label_path = label_dir / f"{image_path.stem}.txt"
            if label_path.exists():
                target_label = out_label_dir / label_path.name
                if not target_label.exists():
                    target_label.write_text(label_path.read_text(encoding="utf-8"), encoding="utf-8")

data_yaml_path = cache_root / "data.yaml"
payload = {
    "path": str(cache_root.resolve()),
    "train": "train/images",
    "val": "valid/images",
    "test": "test/images" if (cache_root / "test" / "images").exists() else "valid/images",
    "nc": int(self.dataset_config["num_classes"]),
    "names": self.dataset_config["class_names"],
}
data_yaml_path.write_text(yaml.safe_dump(payload, sort_keys=False), encoding="utf-8")
return data_yaml_path

def _extract_metrics(self, results: Any) -> dict[str, float]:
    summary = getattr(results, "results_dict", {}) or {}
    box = getattr(results, "box", None)
    metrics = {
        "loss": float(summary.get("val/box_loss", 0.0) + summary.get("val/cls_loss", 0.0) +
summary.get("val/dfl_loss", 0.0)),
        "acc": 0.0,
        "box_iou": 0.0,
        "map50": float(summary.get("metrics/mAP50(B)", 0.0)),
        "map50_95": float(summary.get("metrics/mAP50-95(B)", 0.0)),
        "precision": float(summary.get("metrics/precision(B)", 0.0)),
        "recall": float(summary.get("metrics/recall(B)", 0.0)),
        "params_m": 0.0,
        "gflops": 0.0,
    }

```

```

        "fps": 0.0,
    }
    if box is not None:
        metrics["map50"] = float(getattr(box, "map50", metrics["map50"]))
        metrics["map50_95"] = float(getattr(box, "map", metrics["map50_95"]))
        metrics["precision"] = float(getattr(box, "mp", metrics["precision"]))
        metrics["recall"] = float(getattr(box, "mr", metrics["recall"]))
    return metrics

def notify_target(self) -> Any:
    return SimpleNamespace(run_dir=self.run_dir, best_checkpoint_path=self.best_checkpoint_path)

```

src/losses/__init__.py

```

# -*- coding: utf-8 -*-
"""损失函数模块导出口。"""

from .detection_loss import DetectionLoss

__all__ = ["DetectionLoss"]

```

src/losses/detection_loss.py

```

# -*- coding: utf-8 -*-
"""检测任务损失函数。"""

import torch
from torch import nn

from src.engine.detection_ops import pairwise_iou_xywh

class DetectionLoss(nn.Module):
    """分类损失与匹配框回归损失的组合。"""

    def __init__(
        self,
        box_weight: float = 5.0,
        iou_weight: float = 2.0,
        focal_gamma: float = 2.0,
        background_weight: float = 0.2,
    ) -> None:
        super().__init__()
        self.box_loss = nn.SmoothL1Loss(reduction="none")
        self.box_weight = box_weight
        self.iou_weight = iou_weight
        self.focal_gamma = focal_gamma
        self.background_weight = background_weight

    def forward(
        self,
        predictions: dict[str, torch.Tensor],
        matched_classes: torch.Tensor,
        matched_boxes: torch.Tensor,
        matched_mask: torch.Tensor,
    ) -> dict[str, torch.Tensor]:
        """根据匹配结果计算总损失，并返回训练时常用的中间量。"""
        logits = predictions["logits"]
        pred_boxes = predictions["pred_boxes"]

        batch_size, num_queries, num_classes = logits.shape
        logits_flat = logits.reshape(batch_size * num_queries, num_classes)
        targets_flat = matched_classes.reshape(batch_size * num_queries)
        log_prob = torch.log_softmax(logits_flat, dim=-1)
        prob = log_prob.exp()
        target_log_prob = log_prob.gather(1, targets_flat.unsqueeze(1)).squeeze(1)
        target_prob = prob.gather(1, targets_flat.unsqueeze(1)).squeeze(1)
        background_class = num_classes - 1

```

```

        alpha = torch.ones_like(target_prob)
        alpha = torch.where(targets_flat == background_class, torch.full_like(alpha, self.background_weight),
alpha)
        focal_factor = (1.0 - target_prob).pow(self.focal_gamma)
        classification_loss = -(alpha * focal_factor * target_log_prob).mean()

        valid_pred_boxes = pred_boxes[matched_mask]
        valid_target_boxes = matched_boxes[matched_mask]
        if valid_pred_boxes.numel() == 0:
            box_loss = pred_boxes.sum() * 0.0
            iou_loss = pred_boxes.sum() * 0.0
        else:
            raw_box_loss = self.box_loss(valid_pred_boxes, valid_target_boxes)
            box_loss = raw_box_loss.mean()
            iou_matrix = pairwise_iou_xywh(valid_pred_boxes, valid_target_boxes)
            iou_loss = 1.0 - torch.diag(iou_matrix).mean()

        loss = classification_loss + self.box_weight * box_loss + self.iou_weight * iou_loss
        predicted_class = logits.argmax(dim=-1)
        return {
            "loss": loss,
            "classification_loss": classification_loss.detach(),
            "box_loss": box_loss.detach(),
            "iou_loss": iou_loss.detach(),
            "predicted_class": predicted_class,
            "pred_boxes": pred_boxes.detach(),
        }

```

src/models/__init__.py

```

# -*- coding: utf-8 -*-
"""模型模块导出口。"""

from .qdcn_net import BaselineDetector, QDCRNet

__all__ = ["QDCRNet", "BaselineDetector"]

```

src/models/modules/__init__.py

```

# -*- coding: utf-8 -*-
"""QDCR-Net 子模块导出口。"""

from .cross_residual import CrossResidualBlock
from .quality_aware_fusion import QualityAwareFusion

__all__ = ["CrossResidualBlock", "QualityAwareFusion"]

```

src/models/modules/cross_residual.py

```

# -*- coding: utf-8 -*-
"""跨分支残差交互模块。"""

import math

import torch
from torch import nn

class CrossResidualBlock(nn.Module):
    """对原图分支和增强分支做可配置的双向残差融合。"""

    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        mode: str = "vanilla",

```

```

threshold_init: float = 0.1,
threshold_slope: float = 10.0,
sparse_mode: str = "soft_threshold",
sparse_lambda_init: float = 0.1,
sparse_topk_ratio: float = 0.5,
) -> None:
    super().__init__()
    self.raw_to_enhanced = nn.Linear(in_channels, out_channels)
    self.enhanced_to_raw = nn.Linear(in_channels, out_channels)
    self.activation = nn.ReLU(inplace=True)
    self.mode = mode.lower()
    self.sparse_mode = sparse_mode.lower()
    self.threshold_slope = threshold_slope
    self.sparse_topk_ratio = sparse_topk_ratio

    valid_modes = {"vanilla", "learnable_threshold", "sparse_residual"}
    if self.mode not in valid_modes:
        raise ValueError(f"Unsupported cross residual mode: {mode}")

    valid_sparse_modes = {"soft_threshold", "topk"}
    if self.sparse_mode not in valid_sparse_modes:
        raise ValueError(f"Unsupported sparse residual mode: {sparse_mode}")

    # Use unconstrained parameters and map them to positive thresholds.
    self.raw_threshold = nn.Parameter(torch.full((out_channels,), self._inverse_softplus(threshold_init)))
    self.enhanced_threshold = nn.Parameter(torch.full((out_channels,), self._inverse_softplus(threshold_init)))
    self.raw_sparse_lambda = nn.Parameter(
        torch.full((out_channels,), self._inverse_softplus(sparse_lambda_init))
    )
    self.enhanced_sparse_lambda = nn.Parameter(
        torch.full((out_channels,), self._inverse_softplus(sparse_lambda_init))
    )

def forward(
    self,
    raw_feature: torch.Tensor,
    enhanced_feature: torch.Tensor,
) -> tuple[torch.Tensor, torch.Tensor]:
    """让两个分支互相补充信息，同时保留各自的残差语义。"""
    residual_to_raw = self.enhanced_to_raw(enhanced_feature)
    residual_to_enhanced = self.raw_to_enhanced(raw_feature)

    residual_to_raw = self._filter_residual(
        residual_to_raw,
        self.raw_threshold,
        self.raw_sparse_lambda,
    )
    residual_to_enhanced = self._filter_residual(
        residual_to_enhanced,
        self.enhanced_threshold,
        self.enhanced_sparse_lambda,
    )

    mixed_raw = self.activation(raw_feature + residual_to_raw)
    mixed_enhanced = self.activation(enhanced_feature + residual_to_enhanced)
    return mixed_raw, mixed_enhanced

def _filter_residual(
    self,
    residual: torch.Tensor,
    threshold_param: nn.Parameter,
    sparse_lambda_param: nn.Parameter,
) -> torch.Tensor:
    if self.mode == "vanilla":
        return residual
    if self.mode == "learnable_threshold":
        return self._apply_learnable_threshold(residual, threshold_param)
    return self._apply_sparse_residual(residual, sparse_lambda_param)

def _apply_learnable_threshold(self, residual: torch.Tensor, threshold_param: nn.Parameter) -> torch.Tensor:
    threshold = torch.nn.functional.softplus(threshold_param).unsqueeze(0)

```

```

        gate = torch.sigmoid(self.threshold_slope * (residual.abs() - threshold))
        return residual * gate

    def _apply_sparse_residual(self, residual: torch.Tensor, sparse_lambda_param: nn.Parameter) -> torch.Tensor:
        if self.sparse_mode == "topk":
            return self._apply_topk_mask(residual)
        sparse_lambda = torch.nn.functional.softplus(sparse_lambda_param).unsqueeze(0)
        return torch.sign(residual) * torch.relu(residual.abs() - sparse_lambda)

    def _apply_topk_mask(self, residual: torch.Tensor) -> torch.Tensor:
        channels = residual.size(1)
        keep_count = max(1, min(channels, math.ceil(channels * self.sparse_topk_ratio)))
        topk_indices = residual.abs().topk(k=keep_count, dim=1).indices
        mask = torch.zeros_like(residual)
        mask.scatter_(1, topk_indices, 1.0)
        return residual * mask

    @staticmethod
    def _inverse_softplus(value: float) -> float:
        if value <= 0:
            return -20.0
        return math.log(math.expm1(value))

```

src/models/modules/quality_aware_fusion.py

```

# -*- coding: utf-8 -*-
"""质量感知融合模块。"""

import torch
from torch import nn

class QualityAwareFusion(nn.Module):
    """使用门控权重融合原图分支与增强分支特征。"""

    def __init__(self, channels: int) -> None:
        super().__init__()
        self.gate = nn.Sequential(
            nn.Linear(channels * 2, channels),
            nn.ReLU(inplace=True),
            nn.Linear(channels, channels),
            nn.Sigmoid(),
        )

    def forward(
        self,
        raw_feature: torch.Tensor,
        enhanced_feature: torch.Tensor,
        quality_vector: torch.Tensor | None = None,
    ) -> torch.Tensor:
        """根据双分支特征生成融合门控，并可选叠加质量向量。"""
        gate = self.gate(torch.cat([raw_feature, enhanced_feature], dim=1))
        fused = gate * enhanced_feature + (1.0 - gate) * raw_feature
        if quality_vector is not None:
            fused = fused + quality_vector
        return fused

```

src/models/qdcr_net.py

```

# -*- coding: utf-8 -*-
"""QDCR-Net 与基线检测器定义。"""

from pathlib import Path

import torch
from torch import nn

from .modules import CrossResidualBlock, QualityAwareFusion

```

```

class _ConvBranch(nn.Module):
    """共享卷积分支，把输入图像编码为紧凑的全局特征。"""

    def __init__(self, out_channels: int = 32) -> None:
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(inplace=True),
            nn.Conv2d(16, out_channels, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
        )
        self.spatial_pool = nn.AdaptiveAvgPool2d((8, 8))
        self.global_pool = nn.AdaptiveAvgPool2d((1, 1))

    def forward(self, image: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
        """同时输出空间特征图和全局特征，兼顾定位与全局语义。"""
        feature_map = self.encoder(image)
        spatial_feature = self.spatial_pool(feature_map)
        global_feature = self.global_pool(feature_map).flatten(1)
        return spatial_feature, global_feature

class _DetectionHead(nn.Module):
    """基于空间 token 与 query 的检测头，同时输出类别 logits 与归一化边界框。"""

    def __init__(self, feature_dim: int, num_detection_classes: int) -> None:
        super().__init__()
        self.token_proj = nn.Conv2d(feature_dim, feature_dim, kernel_size=1)
        self.query_proj = nn.Linear(feature_dim, feature_dim)
        self.query_norm = nn.LayerNorm(feature_dim)
        self.classifier = nn.Sequential(
            nn.Linear(feature_dim, feature_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feature_dim, num_detection_classes),
        )
        self.box_size_scale = nn.Sequential(
            nn.Linear(feature_dim, feature_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feature_dim, 2),
            nn.Sigmoid(),
        )
        self.box_center_offset = nn.Sequential(
            nn.Linear(feature_dim, feature_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feature_dim, 2),
            nn.Tanh(),
        )

    def forward(
        self,
        spatial_feature: torch.Tensor,
        global_feature: torch.Tensor,
        query_embedding: torch.Tensor,
    ) -> tuple[torch.Tensor, torch.Tensor]:
        """通过 query 在空间特征图上聚合局部证据，避免退化成固定模板框。"""
        batch_size, _, height, width = spatial_feature.shape
        tokens = self.token_proj(spatial_feature).flatten(2).transpose(1, 2)
        query = self.query_proj(query_embedding).unsqueeze(0).expand(batch_size, -1, -1)
        query = self.query_norm(query + global_feature.unsqueeze(1))
        attention = torch.matmul(query, tokens.transpose(1, 2)) / (tokens.size(-1) ** 0.5)
        attention = attention.softmax(dim=-1)
        query_features = torch.matmul(attention, tokens) + global_feature.unsqueeze(1)
        grid_y, grid_x = torch.meshgrid(
            torch.linspace(0.0, 1.0, steps=height, device=spatial_feature.device, dtype=spatial_feature.dtype),

```

```

        torch.linspace(0.0, 1.0, steps=width, device=spatial_feature.device, dtype=spatial_feature.dtype),
        indexing="ij",
    )
    spatial_coords = torch.stack([grid_x, grid_y], dim=-1).reshape(height * width, 2)
    coords = spatial_coords.unsqueeze(0).expand(batch_size, -1, -1)
    centers = torch.matmul(attention, coords)
    center_offset = self.box_center_offset(query_features) / max(height, width)
    centers = (centers + center_offset).clamp(0.0, 1.0)
    centered_coords = coords.unsqueeze(1) - centers.unsqueeze(2)
    variances = torch.sum(attention.unsqueeze(-1) * centered_coords.pow(2), dim=2).clamp(min=1e-6)
    base_sizes = 2.0 * torch.sqrt(variances)
    size_scale = 0.5 + self.box_size_scale(query_features)
    sizes = (base_sizes * size_scale).clamp(0.02, 1.0)
    pred_boxes = torch.cat([centers, sizes], dim=-1)
    return self.classifier(query_features), pred_boxes

```

```
class QDCRNet(nn.Module):
```

```
    """最小化实现的 QDCR 多 query 检测器。"""
```

```
    def __init__(
```

```

        self,
        num_classes: int,
        feature_dim: int = 32,
        num_queries: int = 8,
        cross_residual_mode: str = "vanilla",
        cross_residual_threshold_init: float = 0.1,
        cross_residual_threshold_slope: float = 10.0,
        cross_residual_sparse_mode: str = "soft_threshold",
        cross_residual_sparse_lambda_init: float = 0.1,
        cross_residual_topk_ratio: float = 0.5,
    ) -> None:

```

```

        super().__init__()
        self.num_classes = num_classes
        self.feature_dim = feature_dim
        self.num_queries = num_queries
        self.num_detection_classes = num_classes + 1

        self.raw_branch = _ConvBranch(out_channels=feature_dim)
        self.enhanced_branch = _ConvBranch(out_channels=feature_dim)
        self.cross_residual = CrossResidualBlock(
            in_channels=feature_dim,
            out_channels=feature_dim,
            mode=cross_residual_mode,
            threshold_init=cross_residual_threshold_init,
            threshold_slope=cross_residual_threshold_slope,
            sparse_mode=cross_residual_sparse_mode,
            sparse_lambda_init=cross_residual_sparse_lambda_init,
            sparse_topk_ratio=cross_residual_topk_ratio,
        )

```

```

        self.fusion = QualityAwareFusion(channels=feature_dim)
        self.query_embed = nn.Embedding(num_queries, feature_dim)
        self.head = _DetectionHead(feature_dim, self.num_detection_classes)

```

```
    def forward(self, raw_image: torch.Tensor, enhanced_image: torch.Tensor) -> dict[str, torch.Tensor]:
```

```

        # 两个分支分别提取特征，再通过交叉残差和质量感知模块融合。
        raw_map, raw_feature = self.raw_branch(raw_image)
        enhanced_map, enhanced_feature = self.enhanced_branch(enhanced_image)
        raw_feature, enhanced_feature = self.cross_residual(raw_feature, enhanced_feature)
        fused_feature = self.fusion(raw_feature, enhanced_feature)
        fusion_gate = torch.sigmoid(fused_feature).unsqueeze(-1).unsqueeze(-1)
        fused_map = fusion_gate * enhanced_map + (1.0 - fusion_gate) * raw_map
        logits, pred_boxes = self.head(fused_map, fused_feature, self.query_embed.weight)
        return {
            "detections": logits,
            "fused_feature": fused_feature,
            "fused_map": fused_map,
            "logits": logits,
            "pred_boxes": pred_boxes,
        }

```

```

def save_checkpoint(
    self,
    path: Path,
    optimizer: torch.optim.Optimizer | None = None,
    metadata: dict | None = None,
) -> None:
    """保存模型、优化器和附加元数据。"""
    payload = {
        "model_state": self.state_dict(),
        "optimizer_state": optimizer.state_dict() if optimizer is not None else None,
        "metadata": metadata or {},
    }
    path.parent.mkdir(parents=True, exist_ok=True)
    torch.save(payload, path)

def load_checkpoint(
    self,
    path: Path,
    optimizer: torch.optim.Optimizer | None = None,
    map_location: str | torch.device = "cpu",
    strict: bool = True,
) -> dict:
    """加载 checkpoint, 并在需要时恢复优化器状态。"""
    payload = torch.load(path, map_location=map_location)
    self.load_state_dict(payload["model_state"], strict=strict)
    if optimizer is not None and payload.get("optimizer_state") is not None:
        optimizer.load_state_dict(payload["optimizer_state"])
    return payload.get("metadata", {})

class BaselineDetector(nn.Module):
    """单分支基线检测器, 接口与 QDCR-Net 保持一致。"""

    def __init__(self, num_classes: int, feature_dim: int = 32, num_queries: int = 8) -> None:
        super().__init__()
        self.num_classes = num_classes
        self.feature_dim = feature_dim
        self.num_queries = num_queries
        self.num_detection_classes = num_classes + 1

        self.branch = _ConvBranch(out_channels=feature_dim)
        self.query_embed = nn.Embedding(num_queries, feature_dim)
        self.head = _DetectionHead(feature_dim, self.num_detection_classes)

    def forward(self, raw_image: torch.Tensor, enhanced_image: torch.Tensor) -> dict[str, torch.Tensor]:
        # 基线模型不使用增强图, 只保留相同的输出协议, 便于统一训练和评估流程。
        del enhanced_image
        spatial_feature, feature = self.branch(raw_image)
        logits, pred_boxes = self.head(spatial_feature, feature, self.query_embed.weight)
        return {
            "detections": logits,
            "fused_feature": feature,
            "fused_map": spatial_feature,
            "logits": logits,
            "pred_boxes": pred_boxes,
        }

    def save_checkpoint(
        self,
        path: Path,
        optimizer: torch.optim.Optimizer | None = None,
        metadata: dict | None = None,
    ) -> None:
        """保存基线模型 checkpoint。"""
        payload = {
            "model_state": self.state_dict(),
            "optimizer_state": optimizer.state_dict() if optimizer is not None else None,
            "metadata": metadata or {},
        }
        path.parent.mkdir(parents=True, exist_ok=True)
        torch.save(payload, path)

```

```

def load_checkpoint(
    self,
    path: Path,
    optimizer: torch.optim.Optimizer | None = None,
    map_location: str | torch.device = "cpu",
    strict: bool = True,
) -> dict:
    """加载基线模型 checkpoint。"""
    payload = torch.load(path, map_location=map_location)
    self.load_state_dict(payload["model_state"], strict=strict)
    if optimizer is not None and payload.get("optimizer_state") is not None:
        optimizer.load_state_dict(payload["optimizer_state"])
    return payload.get("metadata", {})

```

src/utils/__init__.py

```

# -*- coding: utf-8 -*-
"""通用工具模块导出口。"""

from .config import load_config
from .experiment_notify import send_experiment_notification
from .logger import get_logger
from .tb_logger import TensorBoardLogger
from .tracker import ExperimentTracker

__all__ = ["load_config", "get_logger", "send_experiment_notification", "TensorBoardLogger", "ExperimentTracker"]

```

src/utils/config.py

```

# -*- coding: utf-8 -*-
"""配置加载工具。"""

from pathlib import Path

import yaml

def load_config(path: str | Path) -> dict:
    """读取 YAML 配置文件并返回字典。"""
    config_path = Path(path)
    with config_path.open("r", encoding="utf-8") as handle:
        return yaml.safe_load(handle)

```

src/utils/dingtalk_util.py

```

"""钉钉消息通知工具。

支持两类消息：
- Markdown 通知
- ActionCard 正式通知
"""

from __future__ import annotations

from datetime import datetime
import json
from pathlib import Path
import sys
from typing import Any, Dict, Iterable, Optional
from urllib import error, request

PROJECT_ROOT = Path(__file__).resolve().parent.parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

```

```

try:
    from geomatric.logging_config import logger
except ImportError:
    from src.utils.logger import get_logger

    logger = get_logger("qdc.dingtalk")

DEFAULT_ACCESS_TOKEN = ***

def _post_robot_payload(
    access_token: str,
    body: Dict[str, Any],
    timeout: int = 10,
) -> Dict[str, Any]:
    """发送原始钉钉机器人 payload。"""

    url = f"***"
    payload = json.dumps(body).encode("utf-8")
    http_request = request.Request(
        url,
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    try:
        with request.urlopen(http_request, timeout=timeout) as response:
            response_text = response.read().decode("utf-8")
    except error.HTTPError as exc:
        response_text = exc.read().decode("utf-8", errors="replace")
        logger.error("钉钉机器人消息发送失败: %s", response_text)
        raise

    logger.info("钉钉机器人消息响应: %s", response_text)
    return json.loads(response_text)

def send_custom_robot_group_message(
    access_token: str,
    msg: str,
    at_user_ids: Optional[Iterable[str]] = None,
    at_mobiles: Optional[Iterable[str]] = None,
    is_at_all: bool = False,
    title: str = "【通知】",
    timeout: int = 10,
) -> Dict[str, Any]:
    """发送钉钉 Markdown 群消息。

    默认参数:
    - `is_at_all=False`: 默认不 @ 所有人
    - `title="【通知】"`: 默认通知标题
    - `timeout=10`: HTTP 请求超时 10 秒
    """

    formatted_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    body = {
        "at": {
            "isAtAll": str(is_at_all).lower(),
            "atUserIds": list(at_user_ids or []),
            "atMobiles": list(at_mobiles or []),
        },
        "markdown": {
            "title": f"{formatted_time}{title}",
            "text": msg,
        },
        "msgtype": "markdown",
    }
    return _post_robot_payload(access_token=access_token, body=body, timeout=timeout)

```

```
def send_action_card_message(
    access_token: str,
    title: str,
    text: str,
    single_title: str,
    single_url: str,
    timeout: int = 10,
) -> Dict[str, Any]:
    """发送钉钉 ActionCard 正式通知。"""

    body = {
        "msgtype": "actionCard",
        "actionCard": {
            "title": title,
            "text": text,
            "singleTitle": single_title,
            "singleURL": single_url,
        },
    }
    return _post_robot_payload(access_token=access_token, body=body, timeout=timeout)
```

```
def send_release_success_notice(
    project_name: str,
    branch: str,
    deployed_at: str,
    result_text: str,
    detail_url: str,
    access_token: str = DEFAULT_ACCESS_TOKEN,
    notice_title: str = "发布成功通知",
    button_text: str = "查看详情",
    timeout: int = 10,
) -> Dict[str, Any]:
    """发送发布成功的正式通知卡片。
```

该方法生成的 payload 结构与钉钉 `actionCard` 正式通知一致。

```
text = (
    "## 发布成功\n"
    f"**项目**: {project_name}\n\n"
    f"**分支**: {branch}\n\n"
    f"**时间**: {deployed_at}\n\n"
    f"**结果**: {result_text}\n\n"
    "请点击下方按钮查看详情。"
)
return send_action_card_message(
    access_token=access_token,
    title=notice_title,
    text=text,
    single_title=button_text,
    single_url=detail_url,
    timeout=timeout,
)
```

```
def send_to_dingtalk(msg: str, err: bool = False) -> Dict[str, Any]:
    """发送一条带时间戳的钉钉通知。
```

默认使用模块中固化的 access token; `err=True` 时标题切换为错误通知。

```
formatted_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
return send_custom_robot_group_message(
    DEFAULT_ACCESS_TOKEN,
    f"{formatted_time}通知\n{msg}",
    at_user_ids=[],
    at_mobiles=[],
    is_at_all=False,
```

```
        title="【错误】" if err else "【通知】",
    )
```

src/utls/experiment_notify.py

```
# -*- coding: utf-8 -*-
"""训练/评估完成后的钉钉通知辅助工具。"""

from __future__ import annotations

from datetime import datetime
from pathlib import Path
from typing import Any

from .dingtalk_util import send_to_dingtalk
from .logger import get_logger

def infer_dataset_name(config: dict[str, Any]) -> str:
    """从配置中推断数据集名称。"""
    dataset_config = config.get("dataset", {})
    if dataset_config.get("name"):
        return str(dataset_config["name"])
    for key in ("train_root", "val_root"):
        path_value = dataset_config.get(key)
        if not path_value:
            continue
        path = Path(path_value)
        parts = path.parts
        for marker in ("train", "valid", "val"):
            if marker in parts:
                index = parts.index(marker)
                if index > 0:
                    return parts[index - 1]
        if len(parts) >= 2:
            return parts[-2]
    return "dataset"

def fallback_run_name(config: dict[str, Any], start_time: datetime) -> str:
    """在 run_dir 尚未生成时构造一个可区分实验的名称。"""
    experiment_config = config.get("experiment", {})
    model_config = config.get("model", {})
    train_config = config.get("train", {})
    return (
        f"{model_config.get('name', 'model')}_-"
        f"{infer_dataset_name(config)}_"
        f"bs{train_config.get('batch_size', 'na')}_-"
        f"lr{train_config.get('lr', 'na')}_-"
        f"seed{experiment_config.get('seed', 'na')}_-"
        f"{start_time.strftime('%Y%m%d-%H%M%S')}"
    )

def format_duration(start_time: datetime, end_time: datetime) -> str:
    """把运行时长格式化为 `HH:MM:SS`。"""
    seconds = int((end_time - start_time).total_seconds())
    hours, remainder = divmod(seconds, 3600)
    minutes, secs = divmod(remainder, 60)
    return f"{hours:02d}:{minutes:02d}:{secs:02d}"

def send_experiment_notification(
    *,
    config: dict[str, Any],
    project_root: Path,
    trainer: Any,
    start_time: datetime,
    end_time: datetime,
```

```

success: bool,
stage: str,
summary: dict[str, Any] | None = None,
metrics: dict[str, float] | None = None,
error: BaseException | None = None,
) -> None:
    """发送训练或评估的 success/failure 钉钉通知。"""
    logger = get_logger("qdcn.notify")
    experiment_config = config.get("experiment", {})
    model_config = config.get("model", {})
    train_config = config.get("train", {})
    run_dir = getattr(trainer, "run_dir", None)
    run_name = run_dir.name if run_dir is not None else fallback_run_name(config, start_time)
    tb_dir = str(run_dir) if run_dir is not None else "N/A"
    dataset_name = infer_dataset_name(config)
    best_ckpt = str(getattr(trainer, "best_checkpoint_path", "N/A"))
    common_lines = [
        f"## {stage}{'完成' if success else '失败'}",
        f"- 项目名: {project_root.name}",
        f"- 实验名: {experiment_config.get('name', 'unknown')}",
        f"- run_name: {run_name}",
        f"- 模型: {model_config.get('name', 'unknown')}",
        f"- 数据集: {dataset_name}",
        f"- batch_size: {train_config.get('batch_size', 'N/A')}",
        f"- learning_rate: {train_config.get('lr', 'N/A')}",
        f"- epochs: {train_config.get('epochs', 'N/A')}",
        f"- seed: {experiment_config.get('seed', 'N/A')}",
        f"- 开始时间: {start_time.strftime('%Y-%m-%d %H:%M:%S')}",
        f"- {'结束时间' if success else '失败时间': {end_time.strftime('%Y-%m-%d %H:%M:%S')}}",
        f"- 总耗时: {format_duration(start_time, end_time)}",
        f"- TensorBoard: {tb_dir}",
        f"- 最优权重: {best_ckpt}",
    ]
    if success:
        metric_source = metrics or (summary or {}).get("last_val_metrics", {})
        success_lines = [
            f"- loss: {metric_source.get('loss', 'N/A')}",
            f"- acc: {metric_source.get('acc', 'N/A')}",
            f"- mAP: {metric_source.get('map50', 'N/A')}",
            f"- precision: {metric_source.get('precision', 'N/A')}",
            f"- recall: {metric_source.get('recall', 'N/A')}",
        ]
        if summary:
            success_lines.extend(
                [
                    f"- best_epoch: {summary.get('best_epoch', 'N/A')}",
                    f"- best_metric: {summary.get('best_metric', 'N/A')}",
                    f"- monitor: {summary.get('monitor', 'N/A')}",
                ]
            )
        success_lines.append(f"- 说明: {stage}已完成")
        message = "\n".join(common_lines + success_lines)
    else:
        error_name = type(error).__name__ if error is not None else "UnknownError"
        error_message = str(error) if error is not None else "unknown failure"
        failure_lines = [
            f"- 错误摘要: {error_name}: {error_message[:300]}",
            f"- 说明: {stage}执行失败",
        ]
        message = "\n".join(common_lines + failure_lines)

    try:
        send_to_dingtalk(message, err=not success)
        logger.info("[notify] dingtalk %s message sent stage=%s", "success" if success else "failure", stage)
    except Exception as exc:
        logger.warning("[notify] dingtalk message failed stage=%s error=%s", stage, exc)

```

src/utils/logger.py

```
# -*- coding: utf-8 -*-
"""日志工具，统一控制控制台日志格式。"""

import logging

def get_logger(name: str) -> logging.Logger:
    """按名称创建或复用 logger，避免重复挂载 handler。"""
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.INFO)
    handler = logging.StreamHandler()
    handler.setFormatter(logging.Formatter("[%asctime)s %(levelname)s %(name)s: %(message)s"))
    logger.addHandler(handler)
    return logger
```

src/utils/tb_logger.py

```
# -*- coding: utf-8 -*-
"""TensorBoard 日志目录与 writer 封装。"""

from __future__ import annotations

from datetime import datetime
import json
from pathlib import Path
from typing import Any

try:
    from torch.utils.tensorboard import SummaryWriter
except ImportError:
    SummaryWriter = None

class TensorBoardLogger:
    """负责构建独立实验目录并代理 TensorBoard 写入。"""

    def __init__(self, config: dict[str, Any]) -> None:
        self.config = config
        self.project_root = Path(__file__).resolve().parents[2]
        self.run_dir = self._build_run_dir()
        self.run_dir.mkdir(parents=True, exist_ok=False)
        self.hparams_path = self.run_dir / "hparams.json"
        self.writer = SummaryWriter(log_dir=str(self.run_dir)) if SummaryWriter is not None else None
        print(f"TensorBoard log dir: {self.run_dir}")

    def _build_run_dir(self) -> Path:
        root_dir = self.project_root / "runs"
        root_dir.mkdir(parents=True, exist_ok=True)
        run_name = self._run_name()
        run_dir = root_dir / run_name
        if not run_dir.exists():
            return run_dir
        suffix = 1
        while True:
            candidate = root_dir / f"{run_name}_{suffix}"
            if not candidate.exists():
                return candidate
            suffix += 1

    def _run_name(self) -> str:
        model_config = self.config.get("model", {})
        dataset_config = self.config.get("dataset", {})
        train_config = self.config.get("train", {})
```

```

experiment_config = self.config.get("experiment", {})
timestamp = datetime.now().strftime("%Y%m%d-%H%M%S")
parts = [
    timestamp,
    self._sanitize(str(model_config.get("name", experiment_config.get("name", "model")))),
    self._sanitize(self._dataset_name()),
    f"bs{int(train_config.get('batch_size', 0))}",
    f"lr{train_config.get('lr', 'na')}",
]
seed = experiment_config.get("seed")
if seed is not None:
    parts.append(f"seed{seed}")
tag = experiment_config.get("name")
if tag:
    parts.append(self._sanitize(str(tag)))
return "_".join(part for part in parts if part)

def _dataset_name(self) -> str:
    dataset_config = self.config.get("dataset", {})
    dataset_name = dataset_config.get("name")
    if dataset_name:
        return str(dataset_name)

    for key in ("train_root", "val_root"):
        root_value = dataset_config.get(key)
        if not root_value:
            continue
        root = Path(root_value)
        parts = [part for part in root.parts if part not in {"", "/", "data", "datasets", "downloads"}]
        if "train" in parts:
            return parts[parts.index("train") - 1]
        if "valid" in parts:
            return parts[parts.index("valid") - 1]
        if "val" in parts:
            return parts[parts.index("val") - 1]
        if len(parts) >= 2:
            return parts[-2]
    return "dataset"

def _sanitize(self, value: str) -> str:
    sanitized = []
    for char in value:
        if char.isalnum() or char in {"-", "_", "."}:
            sanitized.append(char)
        else:
            sanitized.append("_")
    return "".join(sanitized).strip("_") or "na"

def log_hparams(self, hparams: dict[str, Any], metrics: dict[str, float] | None = None) -> None:
    serialized = {key: self._serialize(value) for key, value in hparams.items()}
    self.hparams_path.write_text(
        json.dumps(serialized, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )
    if self.writer is None:
        return
    if metrics:
        self.writer.add_hparams(serialized, metrics)
        return
    self.writer.add_text("hparams", json.dumps(serialized, indent=2, ensure_ascii=False), 0)

def add_graph(self, model: Any, input_to_model: Any) -> None:
    if self.writer is None:
        return
    self.writer.add_graph(model, input_to_model=input_to_model)

def add_histogram(self, tag: str, values: Any, step: int) -> None:
    if self.writer is None:
        return
    self.writer.add_histogram(tag, values, step)

```

```

def add_scalar(self, tag: str, value: float, step: int) -> None:
    if self.writer is None:
        return
    self.writer.add_scalar(tag, value, step)

def flush(self) -> None:
    if self.writer is not None:
        self.writer.flush()

def close(self) -> None:
    if self.writer is not None:
        self.writer.flush()
        self.writer.close()

def _serialize(self, value: Any) -> str | bool | int | float:
    if isinstance(value, (bool, int, float, str)):
        return value
    if value is None:
        return "none"
    return str(value)

```

src/utils/tracker.py

```

# -*- coding: utf-8 -*-
"""实验跟踪工具，兼容 TSV 与 TensorBoard 两种记录方式。"""

from collections.abc import Iterable
from pathlib import Path
from typing import Any

from .tb_logger import TensorBoardLogger

class ExperimentTracker:
    """负责把训练/评估标量写入磁盘，并在可用时同步到 TensorBoard。"""

    def __init__(self, config: dict[str, Any], run_dir: Path | None = None) -> None:
        self.tb_logger = TensorBoardLogger(config)
        self.run_dir = run_dir or self.tb_logger.run_dir
        self.run_dir.mkdir(parents=True, exist_ok=True)
        self.scalar_path = self.run_dir / "scalars.tsv"

        if not self.scalar_path.exists():
            self.scalar_path.write_text("step\ttag\tvalue\n", encoding="utf-8")

    def log_scalar(self, tag: str, value: float, step: int) -> None:
        """记录单个标量，便于后续画曲线或做回归比较。"""
        with self.scalar_path.open("a", encoding="utf-8") as handle:
            handle.write(f"{step}\t{tag}\t{value:.8f}\n")

        self.tb_logger.add_scalar(tag, value, step)

    def log_hparams(self, hparams: dict[str, Any], metrics: dict[str, float] | None = None) -> None:
        """记录超参数，便于 TensorBoard HParams 面板比较实验。"""
        self.tb_logger.log_hparams(hparams, metrics)

    def log_model_graph(self, model: Any, input_to_model: Any) -> None:
        """在图可追踪时记录模型结构。"""
        self.tb_logger.add_graph(model, input_to_model)

    def log_parameter_histograms(self, named_parameters: Iterable[tuple[str, Any]], step: int, limit: int = 4) -> None:
        """记录少量关键参数的分布，避免 event 文件过大。"""
        logged = 0
        for name, parameter in named_parameters:
            if logged >= limit:
                break
            if parameter is None or not getattr(parameter, "requires_grad", False):
                continue

```

```

        if getattr(parameter, "ndim", 0) < 2:
            continue
        self.tb_logger.add_histogram(f"params/{name}", parameter.detach().cpu(), step)
        logged += 1

def flush(self) -> None:
    """主动刷盘，减少异常退出时的日志丢失。"""
    self.tb_logger.flush()

def close(self) -> None:
    """关闭底层 writer，确保缓存内容落盘。"""
    self.tb_logger.close()

```

configs/base_map_tuned.yaml

```

experiment:
  name: baseline_detector_map_tuned
  seed: 42
  output_dir: outputs/checkpoints/baseline_detector_map_tuned
  runs_dir: runs/baseline_detector_map_tuned
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5

```

configs/base_tuned.yaml

```
experiment:
  name: baseline_detector_tuned
  seed: 42
  output_dir: outputs/checkpoints/baseline_detector_tuned
  runs_dir: runs/baseline_detector_tuned
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/base.yaml

```
experiment:
  name: baseline_detector
  seed: 42
  output_dir: outputs/checkpoints/baseline_detector
  runs_dir: runs/baseline_detector
  device: cuda

dataset:
  train_root: data/datasets/URPC2018/train/images
  val_root: data/datasets/URPC2018/val/images
  enhanced_root: data/datasets/URPC2018_enhanced
  num_classes: 4
  synthetic_size: 16
  max_objects: 8
```

```
class_names:
  - holothurian
  - echinus
  - starfish
  - scallop
image_size: 640
workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 50
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 8
    min_delta: 0.0005

eval:
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_brackish_full.yaml

```
experiment:
  name: baseline_brackish_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_full
  runs_dir: runs/baseline_brackish_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
```

```
batch_size: 4
max_batches_per_epoch: 0
optimizer: adamw
lr: 0.0005
weight_decay: 0.0005
early_stopping:
  enabled: true
  monitor: loss
  mode: min
  patience: 10
  min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_brackish_rf_full.yaml

```
experiment:
  name: baseline_brackish_rf_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_rf_full
  runs_dir: runs/baseline_brackish_rf_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
```

```
conf_thresh: 0.25
iou_thresh: 0.5
```

configs/formal/base_brackish_rf_map_tuned_full.yaml

```
experiment:
  name: baseline_brackish_rf_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_rf_map_tuned_full
  runs_dir: runs/baseline_brackish_rf_map_tuned_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 16
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/base_brackish_rf_tuned_full.yaml

```
experiment:
  name: baseline_brackish_rf_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_rf_tuned_full
  runs_dir: runs/baseline_brackish_rf_tuned_full
  device: cuda

dataset:
  name: brackish_rf
```

```
train_root: data/datasets/downloads/Brackish/train/images
val_root: data/datasets/downloads/Brackish/valid/images
enhanced_root: data/datasets/downloads/Brackish/train/images
num_classes: 6
synthetic_size: 0
max_objects: 16
class_names:
  - crab
  - fish
  - jellyfish
  - shrimp
  - small_fish
  - starfish
image_size: 640
workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/base_ruod_full.yaml

```
experiment:
  name: baseline_ruod_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_full
  runs_dir: runs/baseline_ruod_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: baseline_detector
```

```
feature_dim: 32
num_queries: 8
backbone: single_branch
head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_ruod_map_tuned_full.yaml

```
experiment:
  name: baseline_ruod_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_map_tuned_full
  runs_dir: runs/baseline_ruod_map_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
```

```
min_delta: 0.0005
```

```
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.01  
  iou_thresh: 0.5
```

configs/formal/base_ruod_testb_eval.yaml

```
experiment:  
  name: baseline_ruod_testb_eval  
  seed: 42  
  output_dir: outputs/checkpoints/baseline_ruod_full  
  runs_dir: runs/baseline_ruod_testb_eval  
  device: cuda  
  
dataset:  
  name: ruod_testb  
  train_root: data/datasets/downloads/RUOD/RUOD/train/images  
  val_root: data/datasets/downloads/RUOD/RUOD/testb/images  
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images  
  num_classes: 4  
  synthetic_size: 0  
  max_objects: 8  
  class_names:  
    - holothurian  
    - echinus  
    - starfish  
    - scallop  
  image_size: 320  
  workers: 4  
  
model:  
  name: baseline_detector  
  feature_dim: 32  
  num_queries: 8  
  backbone: single_branch  
  head: detect_proxy  
  
train:  
  epochs: 100  
  batch_size: 4  
  max_batches_per_epoch: 0  
  optimizer: adamw  
  lr: 0.0005  
  weight_decay: 0.0005  
  
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.25  
  iou_thresh: 0.5
```

configs/formal/base_ruod_testc_eval.yaml

```
experiment:  
  name: baseline_ruod_testc_eval  
  seed: 42  
  output_dir: outputs/checkpoints/baseline_ruod_full  
  runs_dir: runs/baseline_ruod_testc_eval  
  device: cuda  
  
dataset:  
  name: ruod_testc  
  train_root: data/datasets/downloads/RUOD/RUOD/train/images  
  val_root: data/datasets/downloads/RUOD/RUOD/testc/images  
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
```

```
num_classes: 4
synthetic_size: 0
max_objects: 8
class_names:
  - holothurian
  - echinus
  - starfish
  - scallop
image_size: 320
workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_ruod_testl_eval.yaml

```
experiment:
  name: baseline_ruod_testl_eval
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_full
  runs_dir: runs/baseline_ruod_testl_eval
  device: cuda

dataset:
  name: ruod_testl
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testl/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
```

```
lr: 0.0005
weight_decay: 0.0005
```

```
eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_ruod_tuned_full.yaml

```
experiment:
  name: baseline_ruod_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_tuned_full
  runs_dir: runs/baseline_ruod_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 4
  prefetch_factor: 2
  cache_preprocessed: true
  cache_dir: /tmp/qdcr_preprocessed/ruod_formal

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
```

```
conf_thresh: 0.05
iou_thresh: 0.5
```

configs/formal/base_uov2_full.yaml

```
experiment:
  name: baseline_uov2_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_uov2_full
  runs_dir: runs/baseline_uov2_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 8
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_uov2_map_tuned_full.yaml

```
experiment:
  name: baseline_uov2_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_uov2_map_tuned_full
  runs_dir: runs/baseline_uov2_map_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
```

```
enhanced_root: data/datasets/downloads/U0v2/train/images
num_classes: 5
synthetic_size: 0
max_objects: 16
class_names:
  - echinus
  - holothurian
  - scallop
  - starfish
  - waterweeds
image_size: 640
workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/base_uov2_tuned_full.yaml

```
experiment:
  name: baseline_uov2_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_uov2_tuned_full
  runs_dir: runs/baseline_uov2_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
```

```
num_queries: 16
backbone: single_branch
head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/faster_rcnn_brackish_rf_full.yaml

```
experiment:
  name: faster_rcnn_brackish_rf_full
  seed: 42
  output_dir: outputs/checkpoints/faster_rcnn_brackish_rf_full
  runs_dir: runs/faster_rcnn_brackish_rf_full
  device: cuda
dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 32
  class_names: [crab, fish, jellyfish, shrimp, small_fish, starfish]
  image_size: 640
  workers: 4
model:
  name: faster_rcnn
train:
  epochs: 20
  batch_size: 2
  max_batches_per_epoch: 0
  lr: 0.005
  weight_decay: 0.0005
eval:
  split: val
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/faster_rcnn_ruod_full.yaml

```
experiment:
  name: faster_rcnn_ruod_full
  seed: 42
  output_dir: outputs/checkpoints/faster_rcnn_ruod_full
  runs_dir: runs/faster_rcnn_ruod_full
  device: cuda
dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
```

```
enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
num_classes: 4
synthetic_size: 0
max_objects: 32
class_names: [holothurian, echinus, starfish, scallop]
image_size: 640
workers: 4
model:
  name: faster_rcnn
train:
  epochs: 20
  batch_size: 2
  max_batches_per_epoch: 0
  lr: 0.005
  weight_decay: 0.0005
eval:
  split: val
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/faster_rcnn_uov2_full.yaml

```
experiment:
  name: faster_rcnn_uov2_full
  seed: 42
  output_dir: outputs/checkpoints/faster_rcnn_uov2_full
  runs_dir: runs/faster_rcnn_uov2_full
  device: cuda
dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 32
  class_names: [echinus, holothurian, scallop, starfish, waterweeds]
  image_size: 640
  workers: 4
model:
  name: faster_rcnn
train:
  epochs: 20
  batch_size: 2
  max_batches_per_epoch: 0
  lr: 0.005
  weight_decay: 0.0005
eval:
  split: val
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_full.yaml

```
experiment:
  name: qdcr_brackish_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_full
  runs_dir: runs/qdcr_brackish_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
```

```
class_names:
  - crab
  - fish
  - jellyfish
  - shrimp
  - small_fish
  - starfish
image_size: 320
workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_rf_full.yaml

```
experiment:
  name: qdcr_brackish_rf_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_rf_full
  runs_dir: runs/qdcr_brackish_rf_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 320
  workers: 4

model:
```

```
name: qdcr_net
feature_dim: 32
num_queries: 8
backbone: dual_branch
quality_aware: true
cross_residual_stages:
  - stage2
  - stage3
small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_rf_map_tuned_full.yaml

```
experiment:
  name: qdcr_brackish_rf_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_rf_map_tuned_full
  runs_dir: runs/qdcr_brackish_rf_map_tuned_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 16
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
```

```
epochs: 120
batch_size: 4
max_batches_per_epoch: 0
optimizer: adamw
lr: 0.0003
weight_decay: 0.0005
early_stopping:
  enabled: true
  monitor: map50
  mode: max
  patience: 20
  min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_rf_tuned_full.yaml

```
experiment:
  name: qdcr_brackish_rf_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_rf_tuned_full
  runs_dir: runs/qdcr_brackish_rf_tuned_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 16
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
```

```
min_delta: 0.0005
```

```
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.05  
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_full.yaml

```
experiment:  
  name: qdcr_ruod_full  
  seed: 42  
  output_dir: outputs/checkpoints/qdcr_ruod_full  
  runs_dir: runs/qdcr_ruod_full  
  device: cuda  
  
dataset:  
  train_root: data/datasets/downloads/RUOD/RUOD/train/images  
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images  
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images  
  num_classes: 4  
  synthetic_size: 0  
  max_objects: 8  
  class_names:  
    - holothurian  
    - echinus  
    - starfish  
    - scallop  
  image_size: 320  
  workers: 4  
  
model:  
  name: qdcr_net  
  feature_dim: 32  
  num_queries: 8  
  backbone: dual_branch  
  quality_aware: true  
  cross_residual_stages:  
    - stage2  
    - stage3  
  small_object_neck: true  
  
train:  
  epochs: 100  
  batch_size: 4  
  max_batches_per_epoch: 0  
  optimizer: adamw  
  lr: 0.001  
  weight_decay: 0.0005  
  early_stopping:  
    enabled: true  
    monitor: loss  
    mode: min  
    patience: 10  
    min_delta: 0.0005  
  
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.25  
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_map_tuned_full.yaml

```
experiment:  
  name: qdcr_ruod_map_tuned_full  
  seed: 42
```

```
output_dir: outputs/checkpoints/qdcr_ruod_map_tuned_full
runs_dir: runs/qdcr_ruod_map_tuned_full
device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_sparse_tuned_full.yaml

```
experiment:
  name: qdcr_ruod_sparse_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_sparse_tuned_full
  runs_dir: runs/qdcr_ruod_sparse_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
```

```
- holothurian
- echinus
- starfish
- scallop
image_size: 640
workers: 1
prefetch_factor: 1
cache_preprocessed: true
cache_dir: /tmp/qdcr_preprocessed/ruod_formal
```

```
model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_mode: sparse_residual
  cross_residual_threshold_init: 0.1
  cross_residual_threshold_slope: 10.0
  cross_residual_sparse_mode: soft_threshold
  cross_residual_sparse_lambda_init: 0.1
  cross_residual_topk_ratio: 0.5
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true
```

```
train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005
```

```
eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_testb_eval.yaml

```
experiment:
  name: qdcr_ruod_testb_eval
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_full
  runs_dir: runs/qdcr_ruod_testb_eval
  device: cuda

dataset:
  name: ruod_testb
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testb/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
```

```
max_objects: 8
class_names:
  - holothurian
  - echinus
  - starfish
  - scallop
image_size: 320
workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.001
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_testc_eval.yaml

```
experiment:
  name: qdcr_ruod_testc_eval
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_full
  runs_dir: runs/qdcr_ruod_testc_eval
  device: cuda

dataset:
  name: ruod_testc
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testc/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true
```

```
train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.001
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_testl_eval.yaml

```
experiment:
  name: qdcr_ruod_testl_eval
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_full
  runs_dir: runs/qdcr_ruod_testl_eval
  device: cuda

dataset:
  name: ruod_testl
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testl/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.001
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_threshold_tuned_full.yaml

```
experiment:
  name: qdcr_ruod_threshold_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_threshold_tuned_full
  runs_dir: runs/qdcr_ruod_threshold_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 1
  prefetch_factor: 1
  cache_preprocessed: true
  cache_dir: /tmp/qdcr_preprocessed/ruod_formal

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_mode: learnable_threshold
  cross_residual_threshold_init: 0.1
  cross_residual_threshold_slope: 10.0
  cross_residual_sparse_mode: soft_threshold
  cross_residual_sparse_lambda_init: 0.1
  cross_residual_topk_ratio: 0.5
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
```

```
conf_thresh: 0.05
iou_thresh: 0.5
```

configs/formal/qdcr_ruod_tuned_full.yaml

```
experiment:
  name: qdcr_ruod_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_tuned_full
  runs_dir: runs/qdcr_ruod_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 1
  prefetch_factor: 1
  cache_preprocessed: true
  cache_dir: /tmp/qdcr_preprocessed/ruod_formal

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_full.yaml

```
experiment:
  name: qdcr_uov2_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_full
  runs_dir: runs/qdcr_uov2_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 8
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 320
  workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_map_probe_v2.yaml

```
experiment:
  name: qdcr_uov2_map_probe_v2
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_map_probe_v2
  runs_dir: runs/qdcr_uov2_map_probe_v2
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
```

```
enhanced_root: data/datasets/downloads/U0v2/train/images
num_classes: 5
synthetic_size: 0
max_objects: 16
class_names:
  - echinus
  - holothurian
  - scallop
  - starfish
  - waterweeds
image_size: 640
workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 20
  batch_size: 4
  max_batches_per_epoch: 120
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 8
    min_delta: 0.000001

eval:
  checkpoint: best
  max_batches: 80
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_map_probe.yaml

```
experiment:
  name: qdcr_uov2_map_probe
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_map_probe
  runs_dir: runs/qdcr_uov2_map_probe
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4
```

```
model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 12
  batch_size: 4
  max_batches_per_epoch: 120
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 6
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 60
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_map_tuned_full.yaml

```
experiment:
  name: qdcr_uov2_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_map_tuned_full
  runs_dir: runs/qdcr_uov2_map_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true
```

```
train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_tuned_full.yaml

```
experiment:
  name: qdcr_uov2_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_tuned_full
  runs_dir: runs/qdcr_uov2_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
```

min_delta: 0.0005

eval:

checkpoint: best

max_batches: 0

conf_thresh: 0.05

iou_thresh: 0.5